



แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอน
และงานวิชาการภาควิชาเทคโนโลยีสารสนเทศ

นายรัฐพิงษ์ ตะเพียนทอง

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย ภาควิชาเทคโนโลยีสารสนเทศ
คณะเทคโนโลยีและการจัดการอุตสาหกรรม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาควิชาเทคโนโลยีสารสนเทศ

นายฐิติพงษ์ ตะเพียนทอง

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย ภาควิชาเทคโนโลยีสารสนเทศ
คณะเทคโนโลยีและการจัดการอุตสาหกรรม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Cluster-Based Platform for Supporting Teaching and Academic Activities
in the Department of Information Technology

MR.THITIPONG TAPIANTHONG

PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE BACHELOR'S DEGREE OF ENGINEERING
PROGRAM IN INFORMATION AND NETWORK ENGINEERING
DEPARTMENT OF INFORMATION TECHNOLOGY
FACULTY OF INDUSTRIAL TECHNOLOGY AND MANAGEMENT
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK
ACADEMIC YEAR 2025
COPYRIGHT OF KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK



ใบรับรองปริญญาโท
คณะเทคโนโลยีและการจัดการอุตสาหกรรม
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เรื่อง แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาควิชาเทคโนโลยี
สารสนเทศ
โดย นายฐิติพงษ์ ตะเพียนทอง

ได้รับอนุมัติให้นับเป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย

_____ คนบดี
(ผู้ช่วยศาสตราจารย์ ดร.กฤษฎากร บุตดาจันทร์)

คณะกรรมการสอบโครงการสหกิจศึกษา

_____ ประธานกรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.ศรายุทธ ธเนศสกุลวัฒนา)

_____ กรรมการ
(อาจารย์ ดร.วัชรชัย คงศิริวัฒนา)

_____ กรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.ชนิษฐา นามิ)

ชื่อ : นายฐิติพงษ์ ตะเพียนทอง
ชื่อปริญญาบัตร : แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงาน
วิชาการภาควิชาเทคโนโลยีสารสนเทศ
สาขาวิชา : วิศวกรรมสารสนเทศและเครือข่าย
: มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
อาจารย์ที่ปรึกษาปริญญาบัตร : ผู้ช่วยศาสตราจารย์ ดร.ชนิษฐา นามิ
ปีการศึกษา : 2568

บทคัดย่อ

ปริญญาบัตร "แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาควิชาเทคโนโลยีสารสนเทศ" ฉบับนี้ มีวัตถุประสงค์เพื่อพัฒนาแพลตฟอร์มคลัสเตอร์เสมือนที่สามารถสร้างและจัดการเครื่องเสมือนลินุกซ์ได้อย่างรวดเร็วและมีประสิทธิภาพ เพื่อสนับสนุนการเรียนการสอนและการทำโครงการในภาควิชาเทคโนโลยีสารสนเทศ และพัฒนาเว็บแอปพลิเคชันแบบ Role-Based Access Control (RBAC) ที่รองรับการใช้งานของทั้งนักศึกษาและอาจารย์ โดยคำนึงถึงความสะดวกในการใช้งาน เสถียรภาพของระบบ และความปลอดภัยในการเข้าถึง ระบบประกอบด้วยสามส่วนหลัก ได้แก่ ส่วนติดต่อผู้ใช้งาน บริการ API สำหรับจัดการการยืนยันตัวตน การจัดการผู้ใช้ และเชื่อมต่อระหว่างส่วนติดต่อผู้ใช้และบริการจัดการเครื่องเสมือน และบริการจัดการเครื่องเสมือนที่เชื่อมต่อกับระบบ Hypervisor เพื่อสร้าง กำหนดค่า และจัดการเครื่องเสมือน ระบบสามารถลดความซับซ้อนในการตั้งค่าสภาพแวดล้อมสำหรับการใช้งานระบบลินุกซ์ เพิ่มประสิทธิภาพการใช้ทรัพยากรเซิร์ฟเวอร์ และสนับสนุนการเรียนการสอนที่เกี่ยวข้องกับระบบเซิร์ฟเวอร์ การพัฒนาแอปพลิเคชัน และการจัดการโครงสร้างพื้นฐานด้านไอทีได้อย่างมีประสิทธิภาพ

(ปริญญาบัตรมีจำนวนทั้งสิ้น 57 หน้า)

Name : Mr.Thitipong Taphanthong
Project Title : Cluster-Based Platform for Supporting Teaching and Academic Activities in the Department of Information Technology
Major Field : Information and Network Engineering
: King Mongkut's University of Technology North Bangkok
Project Advisor : Asst. Prof. Dr.Khanista Namee
Academic Year : 2025

Abstract

This project, "Cluster-Based Platform for Supporting Teaching and Academic Activities in the Department of Information Technology," aims to develop a virtual cluster platform capable of rapidly and efficiently creating and managing Linux virtual machines to support teaching and project work in the Department of Information Technology. The project also focuses on developing a Role-Based Access Control (RBAC) web application that supports both students and faculty, emphasizing ease of use, system stability, and access security. The system consists of three main components: a user interface, an API service for handling authentication and user management and communication between the frontend and VM operations service, and a VM management service that interfaces with the Hypervisor system to create, configure, and manage virtual machines. The system reduces the complexity of setting up Linux environments, improves server resource utilization efficiency, and effectively supports teaching related to server systems, application development, and IT infrastructure management.

(Total 57 pages)

Project Advisor

กิตติกรรมประกาศ

ปริญญานิพนธ์ ”แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาค
วิชาเทคโนโลยีสารสนเทศ” จะไม่สามารถสำเร็จลุล่วงไปได้ด้วยดีหากไม่ได้รับการช่วยเหลือและความ
อนุเคราะห์จากบุคคลหลายท่าน ทางผู้จัดทำขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร.ชนิษฐา นามิ ซึ่งเป็น
อาจารย์ที่ปรึกษา ที่ได้กรุณาให้คำปรึกษาแนะนำ และให้แนวทางในการพัฒนาแพลตฟอร์มคลัสเตอร์และ
การจัดทำปริญญานิพนธ์ รวมทั้งคณาจารย์ภาควิชาเทคโนโลยีสารสนเทศที่ได้ให้ความรู้ด้านเทคโนโลยี
และการพัฒนาระบบให้กับผู้จัดทำเพื่อนำมาประยุกต์ในการพัฒนาโครงการนี้ อีกทั้งขอขอบคุณภาควิชา
เทคโนโลยีสารสนเทศที่ให้การสนับสนุนโครงสร้างพื้นฐานและทรัพยากรที่จำเป็นสำหรับการพัฒนาและ
ทดสอบระบบ

ท้ายที่สุดนี้ ทางผู้จัดทำขอน้อมรำลึกพระคุณบิดา มารดา ผู้ซึ่งมีพระคุณอย่างสูงสุดที่ให้ความ
อุปการะผู้จัดทำมาโดยตลอด รวมทั้งผู้มีพระคุณทุกท่าน คณะผู้จัดทำรู้สึกซาบซึ้งเป็นอย่างยิ่ง จึงใคร่ขอ
ขอบพระคุณเป็นอย่างสูงมา ณ โอกาสนี้ด้วย

ฐิติพงษ์ ตะเพียนทอง

สารบัญ

	หน้า
บทคัดย่อภาษาไทย	ก
บทคัดย่อภาษาอังกฤษ	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
สารบัญภาพ	ฉ
สารบัญตาราง	1
บทที่ 1 บทนำ	1
1.1 ความเป็นมาและความสำคัญ	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของการทำโครงการ	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	3
บทที่ 2 แนวคิด ทฤษฎี และงานวิจัยที่เกี่ยวข้อง	5
2.1 แนวคิดพื้นฐานเกี่ยวกับโครงการ	5
2.2 แนวคิดและทฤษฎีที่เกี่ยวข้องกับแพลตฟอร์มคลัสเตอร์เสมือน	7
2.3 รายงานการค้นคว้า การศึกษา หรืองานวิจัยที่เกี่ยวข้อง	10
บทที่ 3 วิธีการดำเนินงานและการออกแบบระบบ	13
3.1 ขั้นตอนการดำเนินงาน	13
3.2 การศึกษาและวิเคราะห์ความต้องการ	13
3.3 การออกแบบสถาปัตยกรรมระบบ	14
3.4 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา	16
3.5 การวางแผนการพัฒนาและการจัดการโครงการ	17
บทที่ 4 ผลการดำเนินงาน	19
4.1 โครงสร้างโปรเจกต์หลัก	19
4.2 โครงสร้างแอปพลิเคชันและบริการ	20
4.3 สถาปัตยกรรมระบบและเทคโนโลยี	21
4.4 เอกสารประกอบการใช้งาน API และส่วนติดต่อผู้ใช้	22
4.5 ตัวอย่างการใช้งานระบบตามบทบาทผู้ใช้	25
บทที่ 5 สรุปผลการดำเนินงาน	30
5.1 สรุปผลการดำเนินงาน	30
5.2 ปัญหาและอุปสรรค	30
5.3 การแก้ปัญหา	31

สารบัญ (ต่อ)

	หน้า
5.4 ข้อเสนอแนะ	31
บรรณานุกรม	32
ภาคผนวก	34
ภาคผนวก ก: การติดตั้งและใช้งาน Cloud-Based Platform	35
ก.1 การติดตั้งและใช้งาน Cloud-Based Platform	36
ก.2 การเริ่มต้นใช้งานระบบ	36
ก.3 การใช้งานส่วนประกอบต่างๆ ของระบบ	37
ก.4 การตรวจสอบและติดตามระบบ	38
ก.5 การแก้ไขปัญหาเบื้องต้น	39
ภาคผนวก ข: สถาปัตยกรรมระบบและส่วนประกอบทางเทคนิค	41
ข.1 สถาปัตยกรรมระบบและส่วนประกอบทางเทคนิค	42
ข.2 เทคโนโลยีที่ใช้ในการพัฒนา	42
ข.3 Monitoring และ Observability	44
ข.4 Virtualization และ Infrastructure	45
ข.5 Data Validation และ Type Safety	46
ข.6 ข้อดีของสถาปัตยกรรมที่เลือกใช้	46
ภาคผนวก ค: การ Deploy และการตั้งค่าสำหรับ Production	48
ค.1 การ Deploy และการตั้งค่าสำหรับ Production	49
ค.2 การตั้งค่า Environment Variables สำหรับ Production	49
ค.3 การ Deploy ด้วย Docker Compose	50
ค.4 การตั้งค่า Reverse Proxy และ SSL	52
ค.5 การตั้งค่า Database สำหรับ Production	53
ค.6 การตรวจสอบและบำรุงรักษาระบบ	53
ค.7 การแก้ไขปัญหาใน Production	55

สารบัญภาพ

ภาพที่		หน้า
3-1	แผนภาพบริบทของระบบแพลตฟอร์มคลัสเตอร์เสมือน	15
4-1	โครงสร้างโปรเจกต์ทั้งหมด	21
4-2	สถาปัตยกรรมระบบโดยรวม	22
4-3	หน้าเอกสาร API Documentation	23
4-4	หน้าแรกของระบบ (Landing Page)	24
4-5	Dashboard หลักสำหรับนักศึกษา	26
4-6	หน้าจัดการ VM Instances สำหรับนักศึกษา	26
4-7	Dashboard หลักสำหรับอาจารย์และเจ้าหน้าที่	27
4-8	หน้าจัดการหลักสูตรและรายวิชา	28
4-9	หน้าอนุมัติคำขอที่รอดำเนินการ	29
ก-1	คำสั่งสำหรับโคลนโปรเจกต์	36
ก-2	คำสั่งสำหรับติดตั้ง Dependencies	36
ก-3	การตั้งค่า Environment Variables	37
ก-4	การเริ่มต้น Infrastructure Services	37
ก-5	การเริ่มต้น Midori Frontend	37
ก-6	การเริ่มต้น Momoi API Service	38
ก-7	การเริ่มต้น Yuzu VM Operations Service	38
ข-1	สถาปัตยกรรมระบบ Cloud-Based Platform	42
ค-1	การติดตั้งซอฟต์แวร์พื้นฐานสำหรับ Production	49
ค-2	ตัวอย่างการตั้งค่า Environment Variables สำหรับ Production	50
ค-3	ตัวอย่าง Docker Compose สำหรับ Production	51
ค-4	คำสั่งสำหรับ Deploy ด้วย Docker Compose	51
ค-5	การติดตั้ง Nginx	52
ค-6	การตั้งค่า Nginx Reverse Proxy	52
ค-7	การติดตั้ง SSL Certificate	53
ค-8	การตั้งค่า PostgreSQL Database	53
ค-9	การรัน Database Migration	54
ค-10	สคริปต์สำหรับ Database Backup	54
ค-11	สคริปต์ตรวจสอบสถานะระบบ	55
ค-12	การตั้งค่า Cron Jobs	56
ค-13	สคริปต์สำหรับอัปเดตแอปพลิเคชัน	56

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญ

การเรียนการสอนในภาควิชาเทคโนโลยีสารสนเทศมีความจำเป็นต้องใช้ระบบปฏิบัติการลินุกซ์ในหลายรายวิชา โดยเฉพาะในรายวิชาที่เกี่ยวข้องกับระบบเซิร์ฟเวอร์ การพัฒนาแอปพลิเคชัน และการจัดการโครงสร้างพื้นฐานด้านไอที อย่างไรก็ตาม การเตรียมสภาพแวดล้อมสำหรับการใช้งานระบบลินุกซ์นั้นมักมีความยุ่งยากและใช้เวลานาน ทั้งในด้านการติดตั้ง การตั้งค่า และการจัดการทรัพยากร ส่งผลให้การเรียนการสอนไม่สามารถดำเนินไปได้อย่างเต็มประสิทธิภาพ

ที่ผ่านมา นักศึกษาและอาจารย์ต้องใช้วิธีการจำลองระบบลินุกซ์ผ่านเครื่องเสมือน (Virtual Machine), ระบบย่อยของวินโดวส์สำหรับลินุกซ์ (WSL), หรือบริการคลาวด์จากภายนอก ซึ่งแต่ละวิธีมีข้อจำกัด เช่น ความยุ่งยากในการตั้งค่า ความไม่เสถียรของระบบ หรือค่าใช้จ่ายเพิ่มเติมในการใช้งานบริการจากผู้ให้บริการภายนอก โดยเฉพาะในการทำโครงการที่ต้องใช้เซิร์ฟเวอร์ นักศึกษามักต้องจัดหาเครื่องคอมพิวเตอร์ส่วนตัวหรือเซิร์ฟเวอร์ด้วยตนเอง ซึ่งเป็นภาระทั้งด้านเวลาและงบประมาณ

เพื่อแก้ไขปัญหาดังกล่าวข้างต้น โครงการนี้จึงมีแนวคิดในการพัฒนาแพลตฟอร์มคลัสเตอร์ภายในภาควิชา โดยใช้เทคโนโลยี Virtualization ร่วมกับระบบ cloud-init เพื่อสร้างเครื่องจำลองลินุกซ์ที่สามารถใช้งานได้อย่างรวดเร็วและง่ายดาย พร้อมทั้งมีระบบ Web Application สำหรับการจัดการและใช้งานเครื่องเสมือน ทั้งในส่วนของนักศึกษาและอาจารย์

1.2 วัตถุประสงค์ของโครงการ

- 1.2.1 เพื่อออกแบบและพัฒนาแพลตฟอร์มคลัสเตอร์เสมือน (Virtual Cluster Platform) ที่สามารถสร้างและจัดการเครื่องเสมือนลินุกซ์ได้อย่างรวดเร็วและมีประสิทธิภาพ สำหรับสนับสนุนการเรียนการสอนและการทำโครงการในภาควิชาเทคโนโลยีสารสนเทศ
- 1.2.2 เพื่อพัฒนา Web Application แบบ Role-Based Access Control (RBAC) ที่รองรับการใช้งานของทั้งนักศึกษาและอาจารย์ ในการร้องขอ จัดการ และเข้าถึงเครื่องเสมือน โดยคำนึงถึงความสะดวกในการใช้งาน เสถียรภาพของระบบ และความปลอดภัยในการเข้าถึงแพลตฟอร์ม

- 1.2.3 เพื่อเพิ่มประสิทธิภาพการใช้ทรัพยากรเซิร์ฟเวอร์ที่มีอยู่ภายในภาควิชา ผ่านการพัฒนา ระบบคลัสเตอร์แบบรวมศูนย์ ที่สามารถติดตาม วิเคราะห์ และบริหารจัดการทรัพยากร ได้อย่างเหมาะสม โดยมีระบบ Monitoring และ Cloud Storage ที่สามารถรองรับ การใช้งานในลักษณะ multi-user environment

1.3 ขอบเขตของการทำโครงการ

ภาคการศึกษาที่ 1/2568

- 1) จัดทำระบบการจัดการเครื่องคอมพิวเตอร์เซิร์ฟเวอร์
 - 1.1) จัดทำระบบ Hypervisor สำหรับการทำ Virtualization
 - 1.2) ระบบการจัดการเครื่องเสมือนที่อยู่ภายใต้ระบบ Hypervisor ด้วย API
 - 1.2.1) การสร้างเครื่องเสมือนบนระบบ Hypervisor
 - 1.2.2) การแก้ไขข้อมูลเครื่องเสมือนบนระบบ Hypervisor
 - 1.2.3) การลบเครื่องเสมือนบนระบบ Hypervisor
 - 1.2.4) การจัดการสถานะเครื่องเสมือนบนระบบ Hypervisor
- 2) จัดทำระบบ Web Application สำหรับการเข้าใช้งานระบบ
 - 2.1) ระบบเข้าใช้งานสำหรับนักศึกษา
 - 2.1.1) ระบบการยืนยันตัวตนด้วย Google Account เพื่อเข้าใช้งานระบบด้วย โดเมน @email.kmutnb.ac.th และยืนยันภาควิชาที่นักศึกษาสังกัดด้วย รหัสนักศึกษาที่ปรากฏบนที่อยู่อีเมลล์
 - 2.1.2) ระบบยื่นคำร้องขอการใช้งานเครื่องเสมือนสำหรับการใช้งานในการเรียนปกติ หรือสำหรับการทำโครงการพิเศษ
 - 2.1.3) ระบบการจัดการเครื่องเสมือนและการเข้าใช้งานเครื่องเสมือนของนักศึกษา ที่ยังมีสถานะปกติ
 - 2.1.4) ระบบยื่นคำร้องขอต่ออายุการใช้งานเครื่องเสมือนที่มีอยู่เพื่อขอใช้งานเครื่องเสมือนต่อในภาคการศึกษาถัดไป
 - 2.2) ระบบเข้าใช้งานสำหรับอาจารย์หรือผู้ดูแลระบบ
 - 2.2.1) ระบบการยืนยันตัวตนด้วย Google Account เพื่อเข้าใช้งานระบบด้วยโดเมน @itm.kmutnb.ac.th และจำกัดการเข้าใช้ระบบเฉพาะบุคลากรภายในภาควิชา
 - 2.2.2) ระบบจัดการคำร้องขอใช้งานเครื่องเสมือนและคำร้องขอต่ออายุการใช้งานเครื่องเสมือนของนักศึกษา
 - 2.2.3) ระบบการสร้างเครื่องเสมือนสำหรับการใช้งานของอาจารย์หรือผู้ดูแลระบบ
 - 2.2.4) ระบบการจัดการและเข้าใช้งานเครื่องเสมือนของอาจารย์หรือผู้ดูแลระบบ

- 2.2.5) ระบบการจัดการเครื่องเสมือนของนักศึกษาในระบบ โดยสามารถเปลี่ยนแปลงสถานะของเครื่องเสมือนให้อยู่ในรูปแบบการจัดเก็บเครื่องเสมือนถาวรหรือการลบเครื่องเสมือนของนักศึกษา
- 2.2.6) ระบบการกำหนดช่วงเวลาของภาคการศึกษา เพื่อให้ระบบสามารถจัดหมวดหมู่ของเครื่องเสมือนในภาคการศึกษานั้น ๆ ได้
- 2.2.7) ระบบการจัดการรายชื่ออาจารย์หรือผู้ดูแลที่สังกัดภาควิชา

ภาคการศึกษาที่ 2/2568

- 1) จัดทำระบบ Reverse Proxy สำหรับการเข้าใช้งานพอร์ตต่าง ๆ ของเครื่องเสมือนจากระบบเครือข่ายสาธารณะได้
- 2) จัดทำระบบ Cloud Storage สำหรับการจัดเก็บไฟล์สำหรับการใช้งานของอาจารย์หรือผู้ดูแลระบบภายในภาควิชา
- 3) จัดทำระบบ Monitoring ที่จัดเก็บข้อมูล Trace, Metric และ Log โดยสามารถแสดงผลผ่าน Dashboard ได้
- 4) จัดทำการระบบเพิ่มเติมบน Web Application
 - 4.1) ระบบการกำหนดค่า Port Forwarding บนระบบ Reverse Proxy เพื่อกำหนดพอร์ตที่ต้องการใช้งานจากระบบเครือข่ายภายนอก
 - 4.2) ระบบการเข้าใช้งาน Cloud Storage สำหรับอาจารย์หรือผู้ดูแลระบบ เพื่อใช้งานในการจัดเก็บไฟล์
- 5) ประเมินประสิทธิภาพของระบบ
 - 5.1) วัดค่า Provisioning Time วัดค่าเฉลี่ย (Mean) และ ค่ามัธยฐาน (Median) ของเวลาที่ใช้ในการสร้างเครื่องเสมือนใหม่ ตั้งแต่เริ่มต้นกระบวนการจนพร้อมใช้งานเปรียบเทียบกับวิธีการเดิม
 - 5.2) วัดค่า Concurrent User Performance ทดสอบความเร็วการตอบสนองของระบบเมื่อมีผู้ใช้งานพร้อมกันหลายราย วัดค่าการตอบสนองของระบบ (Response Time) เฉลี่ย (ms) และอัตราความล้มเหลว (Error Rate) (%)

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1.4.1 ได้ระบบแพลตฟอร์ม คลัสเตอร์ ที่สามารถใช้งานได้จริง ภายใน ภาค วิชา เทคโนโลยีสารสนเทศ เพื่อสนับสนุนการเรียนการสอน และการทำโครงการของนักศึกษาและอาจารย์
- 1.4.2 ได้ระบบเครื่องเสมือน ลินุกซ์ ที่สามารถสร้าง และ จัดการได้อย่างรวดเร็วผ่าน Web Application โดยไม่ต้องติดตั้งระบบปฏิบัติการเพิ่มเติมบนเครื่องของผู้ใช้งาน

- 1.4.3 ได้ใช้ประโยชน์จากเครื่องเซิร์ฟเวอร์ที่มีอยู่ในภาควิชาอย่างเต็มประสิทธิภาพ โดยนำมาให้บริการในรูปแบบคลัสเตอร์สำหรับการเรียนการสอน
- 1.4.4 ได้โครงการที่สามารถนำไปใช้งานจริงภายในภาควิชา และสามารถต่อยอดพัฒนาเพิ่มเติมในอนาคต
- 1.4.5 ได้โครงการอื่น ๆ ที่สามารถใช้งานได้จริงในภาควิชา โดยนำระบบที่นักศึกษาท่านอื่นที่พัฒนาในโครงการมาติดตั้งบนแพลตฟอร์มคลัสเตอร์

บทที่ 2

แนวคิด ทฤษฎี และงานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดพื้นฐานเกี่ยวกับโครงงาน

2.1.1 ความสำคัญของการเรียนการสอนด้วยระบบปฏิบัติการลินุกซ์ในการศึกษาด้านเทคโนโลยีสารสนเทศ

การเรียนการสอนในสาขาเทคโนโลยีสารสนเทศ โดยเฉพาะในระดับอุดมศึกษา มีความจำเป็นอย่างยิ่งที่จะต้องให้นักศึกษาได้ฝึกฝนการใช้งานระบบปฏิบัติการลินุกซ์ เนื่องจากระบบปฏิบัติการดังกล่าวเป็นพื้นฐานสำคัญในการทำงานด้านเซิร์ฟเวอร์ การพัฒนาระบบ การจัดการโครงสร้างพื้นฐานด้านไอที และเทคโนโลยีคลาวด์ที่กำลังเป็นที่ต้องการของตลาดแรงงานในปัจจุบัน (Anderson, 2008)

การเรียนรู้และฝึกฝนการใช้งานระบบลินุกซ์มีความสำคัญในหลายประการ ได้แก่

- **พัฒนาทักษะการจัดการระบบ** ช่วยให้นักศึกษาเข้าใจและสามารถจัดการระบบปฏิบัติการที่ใช้กันอย่างแพร่หลายในโลกธุรกิจ โดยเฉพาะในด้านเซิร์ฟเวอร์และระบบคลาวด์
- **เสริมสร้างความเข้าใจในการทำงานของระบบคอมพิวเตอร์** การใช้งานลินุกซ์ผ่าน Command Line Interface ช่วยให้นักศึกษาเข้าใจการทำงานของระบบในระดับที่ลึกขึ้น
- **เตรียมความพร้อมสู่การทำงาน** ระบบลินุกซ์เป็นมาตรฐานในอุตสาหกรรมเทคโนโลยี การฝึกฝนการใช้งานจะช่วยให้นักศึกษามีความพร้อมในการทำงานจริง
- **สนับสนุนการเรียนรู้เทคโนโลยีใหม่** หลายเทคโนโลยีสมัยใหม่ เช่น Docker, Kubernetes, และระบบ DevOps ต่างๆ ต้องอาศัยความรู้พื้นฐานเกี่ยวกับลินุกซ์ (Lee & Chen, 2021)

อย่างไรก็ตาม การจัดเตรียมสภาพแวดล้อมสำหรับการเรียนการสอนด้วยระบบลินุกซ์นั้นมักเผชิญกับความท้าทายหลายประการ ได้แก่

- **ความยุ่งยากในการติดตั้งและตั้งค่า** การติดตั้งระบบลินุกซ์บนเครื่องคอมพิวเตอร์ส่วนบุคคลของนักศึกษามักต้องใช้เวลานานและมีความซับซ้อน โดยเฉพาะสำหรับผู้ที่ไม่มีความชำนาญ
- **ข้อจำกัดด้านฮาร์ดแวร์** เครื่องคอมพิวเตอร์ส่วนบุคคลของนักศึกษาอาจมีข้อจำกัดด้านหน่วยความจำหรือพื้นที่จัดเก็บข้อมูล ทำให้การใช้งานเครื่องเสมือนไม่เสถียร
- **ความไม่สม่ำเสมอของสภาพแวดล้อม** นักศึกษาแต่ละคนอาจมีการตั้งค่าและเวอร์ชันของซอฟต์แวร์ที่แตกต่างกัน ทำให้เกิดปัญหาในการเรียนการสอนแบบเดียวกัน
- **การบำรุงรักษาและแก้ไขปัญหา** เมื่อเกิดปัญหาทางเทคนิค อาจารย์และนักศึกษาต้องใช้

เวลามากในการแก้ไขปัญหาแทนที่จะมุ่งเน้นไปที่การเรียนรู้เนื้อหาหลัก

- **ค่าใช้จ่ายในการใช้บริการภายนอก** การใช้บริการคลาวด์จากผู้ให้บริการภายนอกมักมีค่าใช้จ่ายที่สูง โดยเฉพาะเมื่อต้องใช้งานตลอดภาคการศึกษา

2.1.2 บทบาทของเทคโนโลยี Virtualization และ Cloud Computing ในการศึกษา

ในยุคที่เทคโนโลยีสารสนเทศพัฒนาอย่างรวดเร็ว การนำเทคโนโลยี Virtualization และ Cloud Computing มาประยุกต์ใช้ในการศึกษาได้กลายเป็นแนวทางที่สำคัญในการแก้ไขปัญหาและเพิ่มประสิทธิภาพการเรียนการสอน โดยเฉพาะอย่างยิ่งในสาขาเทคโนโลยีสารสนเทศที่ต้องการสภาพแวดล้อมที่หลากหลายและซับซ้อนสำหรับการฝึกปฏิบัติ (Clark & Kwinn, 2016)

เทคโนโลยี Virtualization และ Cloud Computing มีบทบาทสำคัญในการเพิ่มประสิทธิภาพการเรียนการสอนหลายประการ ได้แก่

- **ลดเวลาและความซับซ้อนในการตั้งค่าระบบ** เทคโนโลยีเครื่องเสมือนสามารถสร้างสภาพแวดล้อมที่พร้อมใช้งานได้อย่างรวดเร็ว โดยไม่ต้องให้นักศึกษาแต่ละคนติดตั้งและตั้งค่าระบบด้วยตนเอง
- **เพิ่มความยืดหยุ่นในการจัดการทรัพยากร** สามารถปรับเปลี่ยนขนาดและสเปกของเครื่องเสมือนตามความต้องการของแต่ละรายวิชาหรือโครงการ โดยไม่ต้องลงทุนซื้อฮาร์ดแวร์เพิ่มเติม
- **รองรับการเรียนรู้แบบ Remote Learning** นักศึกษาสามารถเข้าถึงและใช้งานระบบได้จากที่ไหนก็ได้ トラบใดที่มีการเชื่อมต่ออินเทอร์เน็ต ทำให้การเรียนการสอนไม่ถูกจำกัดด้วยสถานที่และเวลา
- **สร้างความสม่ำเสมอของสภาพแวดล้อม** ทุกคนจะได้ใช้งานระบบที่มีการตั้งค่าเหมือนกัน ลดปัญหาที่เกิดจากความแตกต่างของสภาพแวดล้อมในเครื่องของแต่ละคน
- **ประหยัดค่าใช้จ่าย** การใช้ทรัพยากรแบบรวมศูนย์ช่วยลดค่าใช้จ่ายในการซื้อและบำรุงรักษาฮาร์ดแวร์ รวมถึงลดค่าใช้จ่ายสำหรับนักศึกษาในการจัดหาเครื่องมือสำหรับการเรียน

แนวคิดของแพลตฟอร์มคลัสเตอร์สำหรับการศึกษาจึงเป็นการต่อยอดจากข้อดีของเทคโนโลยีเหล่านี้ โดยการสร้างระบบที่สามารถ

- **จัดการเครื่องเสมือนแบบรวมศูนย์** ผ่านระบบ Web Application ที่ใช้งานง่าย ทำให้อาจารย์และนักศึกษาสามารถร้องขอ สร้าง และจัดการเครื่องเสมือนได้โดยไม่ต้องมีความรู้ลึกด้านระบบ
- **ใช้ประโยชน์จากทรัพยากรที่มีอยู่อย่างเต็มที่** การรวมเครื่องเซิร์ฟเวอร์หลายเครื่องเข้าเป็นคลัสเตอร์เดียว ช่วยให้สามารถใช้งานทรัพยากรได้อย่างมีประสิทธิภาพสูงสุด
- **รองรับการใช้งานหลากหลายรูปแบบ** ทั้งการเรียนการสอนปกติ การทำโครงการ การวิจัย

และการพัฒนาระบบต่างๆ ภายในภาควิชา

- **มีระบบติดตามและบริหารจัดการ** ด้วยระบบ Monitoring ที่ช่วยในการติดตามการใช้งาน และแก้ไขปัญหาได้อย่างรวดเร็ว

2.2 แนวคิดและทฤษฎีที่เกี่ยวข้องกับแพลตฟอร์มคลัสเตอร์เสมือน

2.2.1 เทคโนโลยี Virtualization

Virtualization Technology เป็นเทคนิคการจำลองซอฟต์แวร์หรือฮาร์ดแวร์บนซอฟต์แวร์อื่น โดยสภาพแวดล้อมจำลองนี้เรียกว่าเครื่องเสมือน (Virtual Machine) ซึ่งเป็นเทคโนโลยีพื้นฐานสำคัญที่ทำให้สามารถแบ่งทรัพยากรฮาร์ดแวร์เครื่องเดียวออกเป็นหลายๆ ระบบที่ทำงานได้อิสระจากกัน (Garcia & Rodriguez, 2023)

Hypervisor และประเภทของ Virtualization

- **Type 1 Hypervisor (Bare-metal)** เป็น Hypervisor ที่ทำงานโดยตรงบนฮาร์ดแวร์ โดยไม่ต้องผ่านระบบปฏิบัติการหลัก เช่น VMware vSphere, Microsoft Hyper-V, และ Proxmox VE ซึ่งมีประสิทธิภาพสูงและเหมาะสำหรับการใช้งานในระดับองค์กร
- **Type 2 Hypervisor (Hosted)** เป็น Hypervisor ที่ทำงานบนระบบปฏิบัติการหลัก เช่น VMware Workstation, VirtualBox ซึ่งง่ายต่อการใช้งานแต่มีประสิทธิภาพต่ำกว่า Type 1
- **Container-based Virtualization** เป็นรูปแบบของ Virtualization ที่แบ่งปันระบบปฏิบัติการเดียวกัน เช่น Docker, LXC ซึ่งมีความเร็วและประสิทธิภาพสูงกว่าแต่มีความแยกตัวน้อยกว่าเครื่องเสมือนแบบเต็มรูปแบบ

ข้อดีของเทคโนโลยี Virtualization

- **การใช้ทรัพยากรอย่างมีประสิทธิภาพ** สามารถใช้ทรัพยากรฮาร์ดแวร์ได้เต็มประสิทธิภาพ เนื่องจากสามารถรันหลายระบบพร้อมกันได้บนเครื่องเดียว
- **ความยืดหยุ่นในการจัดการ** สามารถสร้าง ลบ หรือปรับแต่งเครื่องเสมือนได้อย่างรวดเร็ว โดยไม่ต้องแตะต้องฮาร์ดแวร์จริง
- **การแยกตัวและความปลอดภัย** แต่ละเครื่องเสมือนทำงานแยกจากกัน หากเครื่องหนึ่งเกิดปัญหาจะไม่ส่งผลกระทบต่อเครื่องอื่น
- **ความสะดวกในการบำรุงรักษา** สามารถสำรองข้อมูล ย้าย หรือกู้คืนระบบได้ง่าย โดยไม่ต้องหยุดการทำงานของเครื่องอื่น

2.2.2 Cloud Computing และ Container Orchestration

Cloud Computing เป็นแนวทางการประมวลผลแบบคลาวด์ที่ช่วยให้สามารถเข้าถึงทรัพยากรการประมวลผลที่กำหนดค่าได้ตามต้องการ ซึ่งสามารถจัดเตรียมและปล่อยทรัพยากรได้อย่างรวดเร็วโดยใช้ความพยายามในการจัดการหรือการโต้ตอบระหว่างผู้ให้บริการน้อยที่สุด โดยมีรูปแบบการให้บริการหลัก ได้แก่

รูปแบบการให้บริการ Cloud Computing

- **Infrastructure as a Service (IaaS)** การให้บริการโครงสร้างพื้นฐานด้านไอที เช่น เครื่องเสมือน พื้นที่จัดเก็บข้อมูล เครือข่าย โดยผู้ใช้สามารถควบคุมระบบปฏิบัติการและแอปพลิเคชันได้
- **Platform as a Service (PaaS)** การให้บริการแพลตฟอร์มสำหรับการพัฒนาและปรับใช้แอปพลิเคชัน โดยไม่ต้องจัดการโครงสร้างพื้นฐานด้านล่าง
- **Software as a Service (SaaS)** การให้บริการซอฟต์แวร์ที่พร้อมใช้งานผ่านอินเทอร์เน็ต โดยไม่ต้องติดตั้งหรือจัดการด้วยตนเอง

Container และ Container Orchestration

Container คือการรวมแอปพลิเคชันและรหัสไบনারี ไลบรารี และการกำหนดค่าเข้าด้วยกันในขณะที่ยังแบ่งปันอิมเมจระบบปฏิบัติการโฮสต์ ดังนั้นคอนเทนเนอร์จึงแบ่งปันทรัพยากรและดำเนินการไมโครเซอร์วิสขนาดเล็ก โปรแกรมซอฟต์แวร์ และแอปพลิเคชันที่ครอบคลุมยิ่งขึ้นได้อย่างมีประสิทธิภาพ ด้วยค่าใช้จ่ายด้านการจัดการน้อยกว่าเครื่องเสมือน

- **ข้อดีของ Container** มีขนาดเล็ก เริ่มต้นเร็ว ใช้ทรัพยากรน้อย และสามารถพกพาได้ง่ายเนื่องจากบรรจุทุกสิ่งที่จำเป็นสำหรับการทำงานของแอปพลิเคชัน
- **Docker** เป็นแพลตฟอร์ม Container ที่ได้รับความนิยมสูงสุด ช่วยให้การสร้าง จัดการ และปรับใช้ Container เป็นเรื่องง่าย

Container Orchestration คือการปรับใช้และจัดการแอปพลิเคชันที่เป็น Container ที่อยู่ในคลัสเตอร์ที่เชื่อมต่อถึงกัน โดยมีโหนดที่คอยควบคุมการทำงานที่เรียกว่าโหนดมาสเตอร์ (Master node) ซึ่งมีหน้าที่ดูแลคลัสเตอร์และจัดการการปรับใช้คอนเทนเนอร์บนโหนดเวิร์กเกอร์ (Worker node)

- **การจัดการทรัพยากรอัตโนมัติ** ระบบจะจัดสรรทรัพยากรให้กับ Container ตามความต้องการและสถานะของคลัสเตอร์
- **High Availability** สามารถกระจาย Container ไปยังหลายโหนดเพื่อป้องกันการหยุดทำงานเมื่อโหนดใดโหนดหนึ่งเกิดปัญหา
- **Auto Scaling** สามารถปรับขนาดของแอปพลิเคชันขึ้นลงอัตโนมัติตามปริมาณการใช้งาน
- **Service Discovery และ Load Balancing** ช่วยในการเชื่อมต่อ Container ต่างๆ และ

กระจายโหลดการทำงาน

2.2.3 Web Application และเทคโนโลยีสนับสนุน

Web Application คือการพัฒนาระบบงานบนเว็บ ข้อมูลต่างๆ ในระบบสามารถเข้าถึงได้ทั้งเครือข่ายภายในหรือเครือข่ายสาธารณะ ซึ่งทำให้ใช้งานง่ายผ่านเว็บเบราว์เซอร์ โดยในการใช้งานนั้นไม่จำเป็นต้องติดตั้งโปรแกรมใดๆ เพิ่มเติม ทำให้เหมาะสำหรับการใช้งานในสภาพแวดล้อมที่มีผู้ใช้งานหลากหลาย

คุณสมบัติสำคัญของ Web Application สำหรับการจัดการระบบคลัสเตอร์

- **Cross-platform Compatibility** สามารถใช้งานได้บนระบบปฏิบัติการต่างๆ โดยไม่ต้องพัฒนาแยกสำหรับแต่ละแพลตฟอร์ม
- **Centralized Management** ช่วยให้อาจจัดการระบบจากจุดเดียว ลดความซับซ้อนในการดูแลระบบหลายเครื่อง
- **Role-Based Access Control (RBAC)** สามารถกำหนดสิทธิ์การเข้าถึงตามบทบาทของผู้ใช้ เช่น นักศึกษา อาจารย์ และผู้ดูแลระบบ
- **Real-time Monitoring** สามารถแสดงสถานะของระบบแบบเรียลไทม์ ช่วยในการติดตามและแก้ไขปัญหาได้รวดเร็ว

Cloud-Init และการตั้งค่าเครื่องเสมือนอัตโนมัติ

Cloud-Init คือการตั้งค่าที่ถูกรับรองไว้ล่วงหน้าสำหรับการจัดเตรียมเครื่องจำลองใหม่โดยอัตโนมัติ ซึ่งรวมถึงการสร้างข้อมูลประจำตัวผู้ใช้และการตั้งค่าระบบขั้นพื้นฐาน ด้วยการสร้างไฟล์ cloud-config ที่ปรับให้เหมาะกับเครื่องเสมือนแต่ละเครื่องอย่างไดนามิก

- **การตั้งค่าผู้ใช้งานอัตโนมัติ** สามารถสร้างบัญชีผู้ใช้ กำหนดรหัสผ่าน และตั้งค่า SSH keys ได้โดยอัตโนมัติ
- **การติดตั้งซอฟต์แวร์เริ่มต้น** สามารถกำหนดให้ติดตั้งแพ็คเกจต่างๆ ที่จำเป็นสำหรับการใช้งานเบื้องต้น
- **การตั้งค่าเครือข่าย** สามารถกำหนดค่า IP Address, DNS, และการตั้งค่าเครือข่ายอื่นๆ ได้
- **การรันสคริปต์เริ่มต้น** สามารถกำหนดให้รันคำสั่งหรือสคริปต์ต่างๆ หลังจากบูตระบบขึ้นมาแล้ว

Reverse Proxy และการจัดการการเข้าถึง

Reverse Proxy คือตัวกลางที่ส่งข้อมูลเครือข่ายที่แลกเปลี่ยนข้อมูลระหว่างไคลเอนต์และเซิร์ฟเวอร์ โดยจะรับคำขอจากไคลเอนต์บนเครือข่ายสาธารณะ (อินเทอร์เน็ต) ส่งต่อคำขอไปยังเซิร์ฟเวอร์ในเครือข่ายภายใน จากนั้นจึงส่งคำตอบที่สร้างโดยเซิร์ฟเวอร์ไปยังไคลเอนต์

- **Port Forwarding** ช่วยให้สามารถเข้าถึงบริการต่างๆ ในเครื่องเสมือนจากภายนอกได้โดยผ่านพอร์ตที่กำหนด
- **Load Balancing** สามารถกระจายโหลดไปยังเซิร์ฟเวอร์หลายตัวเพื่อเพิ่มประสิทธิภาพ
- **SSL Termination** สามารถจัดการการเข้ารหัส SSL/TLS ได้ที่ตัว Proxy โดยไม่ต้องตั้งค่าในแต่ละเซิร์ฟเวอร์
- **Security** ช่วยป้องกันการเข้าถึงโดยตรงไปยังเซิร์ฟเวอร์ภายใน และสามารถกำหนดกฎการเข้าถึงได้

สรุปได้ว่า การนำเทคโนโลยี Virtualization, Cloud Computing, Container Orchestration, Web Application และเทคโนโลยีสนับสนุนต่างๆ มาใช้ร่วมกันในการพัฒนาแพลตฟอร์มคลัสเตอร์สำหรับการศึกษา จะช่วยให้สามารถสร้างสภาพแวดล้อมการเรียนการสอนที่มีประสิทธิภาพ ยืดหยุ่น และตอบสนองความต้องการของผู้ใช้งานได้หลากหลายรูปแบบ โดยเฉพาะอย่างยิ่งในสาขาเทคโนโลยีสารสนเทศที่ต้องการการฝึกปฏิบัติในสภาพแวดล้อมที่เสมือนจริง

2.3 รายงานการค้นคว้า การศึกษา หรืองานวิจัยที่เกี่ยวข้อง

2.3.1 การพัฒนาระบบ Virtualization สำหรับการศึกษา

การศึกษาวิจัยเรื่อง ”Guide to Security for Full Virtualization Technologies” โดย Karen Scarfone, Murugiah Souppaya, และ Paul Hoffman ได้นำเสนอแนวทางและมาตรฐานด้านความปลอดภัยสำหรับเทคโนโลยี Virtualization แบบเต็มรูปแบบ (Karen Scarfone, 2011). งานวิจัยนี้มีความสำคัญต่อการพัฒนาแพลตฟอร์มคลัสเตอร์เนื่องจากให้ข้อมูลเกี่ยวกับการจำแนกประเภทของ Virtualization ตามชั้นสถาปัตยกรรมคอมพิวเตอร์ และแนวทางการรักษาความปลอดภัยที่จำเป็นซึ่งเป็นองค์ความรู้พื้นฐานสำคัญในการออกแบบระบบที่ปลอดภัยและเหมาะสมสำหรับสภาพแวดล้อมการศึกษา

2.3.2 การประยุกต์ใช้ Cloud Computing ในการศึกษา

Danairat T. ได้นำเสนอเอกสารเรื่อง ”Cloud Computing Technology The Architecture Overview” ซึ่งให้ภาพรวมของสถาปัตยกรรม Cloud Computing และแนวคิดการประมวลผลแบบคลาวด์ที่ช่วยให้สามารถเข้าถึงทรัพยากรการประมวลผลที่กำหนดค่าได้ตามต้องการ (T., 2015). งานศึกษานี้เป็นแนวทางสำคัญในการทำความเข้าใจรูปแบบการให้บริการ เช่น IaaS, PaaS, และ SaaS ซึ่งเป็นแนวคิดที่นำมาประยุกต์ใช้ในการพัฒนาแพลตฟอร์มคลัสเตอร์เพื่อให้บริการเครื่องเสมือนแก่ผู้ใช้งานในสถาบันการศึกษา

2.3.3 การพัฒนาระบบติดตามความก้าวหน้าสำหรับเอกสารโครงการพิเศษ

Rawiporn Suamsiri และ Kamonwan Janmanee ได้พัฒนาระบบ **"Progress Tracking System for Special Project Documents"** ซึ่งเป็นระบบ Web Application ที่ช่วยในการติดตามความก้าวหน้าของการจัดทำเอกสารโครงการพิเศษ (Suamsiri & Janmanee, 2024). งานวิจัยนี้แสดงให้เห็นถึงแนวทางการพัฒนา Web Application ที่ใช้งานง่ายผ่านเว็บเบราว์เซอร์โดยไม่จำเป็นต้องติดตั้งโปรแกรมเพิ่มเติม ซึ่งเป็นแนวคิดที่สอดคล้องกับการพัฒนาส่วนติดต่อผู้ใช้งานสำหรับแพลตฟอร์มคลัสเตอร์ในโครงการนี้

2.3.4 การใช้งาน Container Technologies ในสถาปัตยกรรมต่างๆ

งานวิจัยของ SHAHIDULLAH KAISER, MD.SADUNHAQ, ALI AMAN TOSUN, และ TURGAY KORKMAZ เรื่อง **"Container Technologies for ARM Architecture: A Comprehensive Survey of the State-of-the-Art"** นำเสนอการสำรวจเทคโนโลยี Container บนสถาปัตยกรรม ARM อย่างครอบคลุม (Kaiser et al., 2022). แสดงให้เห็นถึงการพัฒนาและการใช้งาน Container ที่มีประสิทธิภาพสูง ใช้ทรัพยากรน้อย และสามารถดำเนินการไมโครเซอร์วิสได้อย่างมีประสิทธิภาพ ซึ่งเป็นข้อมูลสำคัญในการออกแบบระบบที่ใช้ Container เป็นส่วนหนึ่งของโครงสร้างพื้นฐาน

2.3.5 Container Orchestration ในอุตสาหกรรมยานยนต์

Naresh Nayak, Dennis Grewe, และ Sebastian Schildt ได้นำเสนอการศึกษาเรื่อง **"Automotive Container Orchestration: Requirements, Challenges and Open Directions"** ซึ่งอธิบายแนวคิดของ Container Orchestration ในการปรับใช้และจัดการแอปพลิเคชันที่เป็น Container ในคลัสเตอร์ (Nayak et al., 2023). งานวิจัยนี้ให้ความรู้เกี่ยวกับการทำงานของโหนดมาสเตอร์และโหนดเวิร์กเกอร์ รวมถึงคุณสมบัติสำคัญ เช่น High Availability, Auto Scaling, และ Load Balancing ซึ่งเป็นแนวคิดที่นำมาประยุกต์ใช้ในการพัฒนาระบบจัดการคลัสเตอร์ในโครงการนี้

2.3.6 การสร้างโครงสร้างพื้นฐานคลาวด์สำหรับการจัดตารางเครื่องเสมือน

Jonathan Chya และ Xunfei Jiang ได้นำเสนอการศึกษาเรื่อง **"Building a Cloud Infrastructure for Virtual Machine Scheduling in Datacenters"** ซึ่งเป็นการศึกษาเกี่ยวกับการสร้างโครงสร้างพื้นฐานคลาวด์สำหรับการจัดตารางเครื่องเสมือนในศูนย์ข้อมูล รวมถึงการใช้ Cloud-Init ในการจัดเตรียมเครื่องเสมือนใหม่โดยอัตโนมัติ (Chya & Jiang, 2024). งานวิจัยนี้แสดงให้เห็นถึงการใช้ไฟล์ cloud-config ที่ปรับให้เหมาะกับเครื่องเสมือนแต่ละเครื่องอย่างไดนามิก ซึ่งช่วยเพิ่มประสิทธิภาพด้านพลังงาน ความเร็ว และการปรับแต่งกระบวนการจัดเตรียมเครื่องเสมือนได้อย่างมาก

2.3.7 การพัฒนาสถาปัตยกรรม Reverse Proxy

Yu TAO และ Gang CHEN ได้เผยแพร่งานวิจัยเรื่อง **”An Extensible Universal Reverse Proxy Architecture”** ซึ่งนำเสนอสถาปัตยกรรม Reverse Proxy ที่มีความยืดหยุ่นและสามารถขยายได้ (Tao & Chen, 2016). งานวิจัยนี้อธิบายการทำงานของ Reverse Proxy ในการรับคำขอจากไคลเอนต์บนเครือข่ายสาธารณะและส่งต่อไปยังเซิร์ฟเวอร์ในเครือข่ายภายใน รวมถึงความสามารถในการแคชทรัพยากรเพื่อเพิ่มประสิทธิภาพ ซึ่งเป็นแนวคิดสำคัญที่นำมาใช้ในการพัฒนาระบบ Port Forwarding และการจัดการการเข้าถึงเครื่องเสมือนจากภายนอกในโครงงานนี้

บทที่ 3

วิธีการดำเนินงานและการออกแบบระบบ

3.1 ขั้นตอนการดำเนินงาน

การพัฒนาแพลตฟอร์มคลัสเตอร์เสมือนสำหรับการศึกษามีขั้นตอนการดำเนินงานที่เป็นระบบ โดยแบ่งออกเป็น 5 ขั้นตอนหลัก ดังนี้

- 1) **การศึกษาและวิเคราะห์ความต้องการ** การสำรวจความต้องการของผู้ใช้งาน การศึกษาระบบที่มีอยู่เดิม และการกำหนดข้อกำหนดของระบบ
- 2) **การออกแบบสถาปัตยกรรมระบบ** การออกแบบโครงสร้างระบบทั้งหมด การเลือกเทคโนโลยีที่เหมาะสม และการออกแบบฐานข้อมูล
- 3) **การพัฒนาระบบ** การพัฒนาส่วนต่าง ๆ ของระบบตามสถาปัตยกรรมที่ออกแบบไว้
- 4) **การทดสอบระบบ** การทดสอบการทำงานของระบบ การทดสอบประสิทธิภาพ และการทดสอบความปลอดภัย
- 5) **การประเมินผลและปรับปรุง** การประเมินผลการใช้งานจริง และการปรับปรุงระบบตามผลการประเมิน

3.2 การศึกษาและวิเคราะห์ความต้องการ

3.2.1 การสำรวจความต้องการผู้ใช้งาน

ผู้จัดทำได้ดำเนินการสำรวจความต้องการจากกลุ่มผู้ใช้งานในภาควิชาเทคโนโลยีสารสนเทศ ได้แก่

- **อาจารย์** ต้องการระบบจัดการเครื่องเสมือนที่ใช้งานง่าย สำหรับการสอนและการวิจัย
- **นักศึกษา** ต้องการเข้าถึงเครื่องเสมือนสำหรับการเรียนและทำโครงงาน
- **ผู้ดูแลระบบ** ต้องการเครื่องมือติดตามและจัดการระบบที่มีประสิทธิภาพ

3.2.2 การวิเคราะห์ระบบเดิม

จากการศึกษาระบบที่มีอยู่เดิม พบปัญหาและข้อจำกัดดังนี้

- **การใช้งานที่ซับซ้อน** ต้องใช้ความรู้ทางเทคนิคสูงในการจัดการเครื่องเสมือน
- **ขาดระบบจัดการผู้ใช้** ไม่มีการแยกสิทธิ์การใช้งานตามบทบาท
- **ประสิทธิภาพต่ำ** การสร้างและลบเครื่องเสมือนใช้เวลานาน

- ขาดการติดตาม ไม่มีระบบ Monitoring การใช้ทรัพยากร

3.2.3 ข้อกำหนดของระบบ

จากการวิเคราะห์ความต้องการ ได้กำหนดข้อกำหนดของระบบดังนี้

ข้อกำหนดเชิงหน้าที่ (Functional Requirements)

- ระบบต้องสามารถสร้าง แก้ไข และลบเครื่องเสมือนได้
- ระบบต้องรองรับการจัดการผู้ใช้แบบ Role-Based Access Control
- ระบบต้องมีระบบยืนยันตัวตนผ่าน OAuth 2.0
- ระบบต้องสามารถติดตามการใช้ทรัพยากรแบบ Real-time
- ระบบต้องส่งการแจ้งเตือนเมื่อมีการเปลี่ยนแปลงสถานะ

ข้อกำหนดที่ไม่เกี่ยวกับหน้าที่ (Non-Functional Requirements)

- ประสิทธิภาพ การสร้างเครื่องเสมือนต้องใช้เวลาไม่เกิน 3 นาที
- ความพร้อมใช้งาน ระบบต้องพร้อมใช้งานอย่างน้อย 99.5
- ความปลอดภัย ข้อมูลต้องได้รับการเข้ารหัสและจัดเก็บอย่างปลอดภัย
- ความยืดหยุ่น ระบบต้องรองรับการขยายทรัพยากรได้ในอนาคต

3.3 การออกแบบสถาปัตยกรรมระบบ

3.3.1 สถาปัตยกรรมระบบโดยรวม

ระบบแพลตฟอร์มคลัสเตอร์เสมือนได้รับการออกแบบในรูปแบบ Microservices Architecture โดยแบ่งเป็นส่วนประกอบหลัก 3 ส่วน คือ

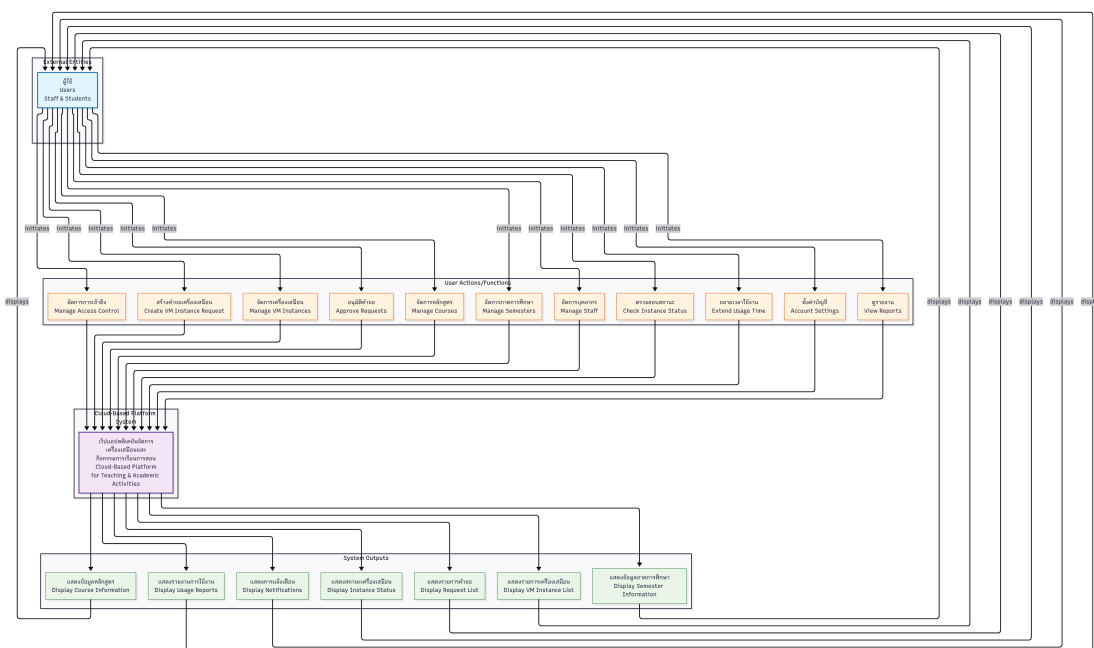
- **Frontend** เว็บแอปพลิเคชันสำหรับผู้ใช้งาน พัฒนาด้วย Next.js และ React
- **Backend** เซิร์ฟเวอร์ API สำหรับการจัดการข้อมูลและการยืนยันตัวตน พัฒนาด้วย Elysia.js
- **Instance Management** เซอร์วิสจัดการเครื่องเสมือนและการสื่อสารกับ Proxmox VE

จากภาพที่ 3-1 แผนภาพบริบทข้างต้นแสดงให้เห็นถึงการเชื่อมต่อและการสื่อสารระหว่างส่วนประกอบต่าง ๆ ของระบบ โดยผู้ใช้งานสามารถเข้าถึงระบบผ่านเว็บแอปพลิเคชัน ที่จะสื่อสารกับ Backend API และระบบจัดการเครื่องเสมือนเพื่อให้บริการที่สมบูรณ์

3.3.2 การออกแบบฐานข้อมูล

ระบบใช้ PostgreSQL เป็นฐานข้อมูลหลัก โดยออกแบบตารางหลักดังนี้

- **User Table** เก็บข้อมูลผู้ใช้งาน บทบาท และสิทธิ์การเข้าถึง
- **Instance Table** เก็บข้อมูลเครื่องเสมือน การกำหนดค่า และสถานะ



ภาพที่ 3-1 แผนภาพบริบทของระบบแพลตฟอร์มคลัสเตอร์เสมือน

- **Template Table** เก็บข้อมูล Template สำหรับการสร้างเครื่องเสมือน
- **Activity Log Table** เก็บประวัติการใช้งานและการเปลี่ยนแปลง
- **Resource Usage Table** เก็บข้อมูลการใช้ทรัพยากรแบบ Time-series

3.3.3 การออกแบบ API และการสื่อสาร

ระบบใช้ RESTful API เป็นหลักในการสื่อสารระหว่างส่วนประกอบต่าง ๆ โดยมีการออกแบบดังนี้

- **Authentication Endpoints** สำหรับการ Login, Logout และ Token Management
- **Instance Management Endpoints** สำหรับการจัดการเครื่องเสมือน (CRUD Operations)
- **Monitoring Endpoints** สำหรับการติดตามการใช้ทรัพยากร

3.3.4 การออกแบบระบบ Message Queue

เพื่อรองรับการประมวลผลแบบ Asynchronous ระบบใช้ RabbitMQ เป็น Message Broker โดยออกแบบ Queue หลักดังนี้

- **Instance Creation Queue** สำหรับงานการสร้างเครื่องเสมือน
- **Instance Deletion Queue** สำหรับงานการลบเครื่องเสมือน

3.4 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา

3.4.1 เทคโนโลยี Frontend

Next.js 14 เป็น React Framework ที่เลือกใช้เนื่องจากคุณสมบัติดังนี้

- **App Router** สำหรับการจัดการ Routing แบบใหม่
- **Server Components** ลดปริมาณ JavaScript ที่ส่งไปยัง Client
- **Static Site Generation** เพิ่มประสิทธิภาพการโหลดหน้าเว็บ
- **Built-in TypeScript Support** รองรับ Type Safety

Tailwind CSS ใช้สำหรับการจัดการ Styling เนื่องจาก

- **Utility-first** เขียน CSS ได้เร็วและมีประสิทธิภาพ
- **Responsive Design** รองรับการแสดงผลบนอุปกรณ์ต่าง ๆ

3.4.2 เทคโนโลยี Backend

Elysia.js เป็น Web Framework ที่ใช้กับ Bun Runtime โดยมีเหตุผลในการเลือกใช้ดังนี้

- **High Performance** มีประสิทธิภาพสูงกว่า Node.js Framework ทั่วไป
- **Type Safety** รองรับ TypeScript แบบเต็มรูปแบบ
- **OpenAPI Integration** สร้าง API Documentation อัตโนมัติ
- **Plugin Ecosystem** มี Plugin หลากหลายสำหรับการพัฒนา

Prisma ORM ใช้สำหรับการจัดการฐานข้อมูล เนื่องจาก

- **Type-safe Database Client** ป้องกันข้อผิดพลาดในการเขียน Query
- **Database Migrations** จัดการการเปลี่ยนแปลง Schema อย่างเป็นระบบ
- **Database Introspection** สร้าง Schema จากฐานข้อมูลที่มีอยู่

3.4.3 Infrastructure และ DevOps

Docker ใช้สำหรับการ Containerization ของแอปพลิเคชัน เพื่อ

- **Environment Consistency** สภาพแวดล้อมการพัฒนาและ Production เหมือนกัน
- **Easy Deployment** ง่ายต่อการ Deploy และ Scale
- **Resource Isolation** แยกทรัพยากรระหว่างแอปพลิเคชัน

Docker Compose ใช้สำหรับการจัดการ Multi-container Application

- **Service Orchestration** จัดการการเริ่มต้นและหยุดทำงานของ Service
- **Network Management** จัดการเครือข่ายระหว่าง Container
- **Volume Management** จัดการการแชร์ข้อมูลระหว่าง Container

3.4.4 เครื่องมือการพัฒนา

Version Control และ Collaboration

- **Git** สำหรับการควบคุมเวอร์ชันของโค้ด
- **GitHub** สำหรับการเก็บโค้ดและการทำงานร่วมกัน
- **GitHub Actions** สำหรับ CI/CD Pipeline

Code Quality และ Testing

- **ESLint และ Prettier** สำหรับการตรวจสอบและจัดรูปแบบโค้ด
- **TypeScript** สำหรับ Static Type Checking
- **Jest** สำหรับ Unit Testing

3.5 การวางแผนการพัฒนาและการจัดการโครงการ

3.5.1 วิธีการพัฒนา (Development Methodology)

โครงการนี้ใช้วิธีการพัฒนาแบบ Agile Development โดยแบ่งการพัฒนาเป็น Sprint ขนาด 2 สัปดาห์ แต่ละ Sprint จะมีขั้นตอนดังนี้

- 1) **Sprint Planning** วางแผนงานที่จะทำใน Sprint นั้น ๆ
- 2) **Daily Standup** ติดตามความคืบหน้าและปัญหาที่เกิดขึ้น
- 3) **Development** การพัฒนาตาม User Stories ที่กำหนด
- 4) **Testing** การทดสอบฟีเจอร์ที่พัฒนาเสร็จ
- 5) **Sprint Review** ทบทวนผลงานและรับ Feedback
- 6) **Sprint Retrospective** หาจุดปรับปรุงสำหรับ Sprint ถัดไป

3.5.2 แผนการพัฒนาตามระยะเวลา

การพัฒนาแบ่งออกเป็น 4 Phase หลัก ตามลำดับดังนี้

Phase 1: Foundation (สัปดาห์ที่ 1-4)

- ตั้งค่า Development Environment
- พัฒนาระบบ Authentication และ User Management
- สร้าง Database Schema และ API พื้นฐาน
- พัฒนา UI Components พื้นฐาน

Phase 2: Core Features (สัปดาห์ที่ 5-8)

- พัฒนาระบบจัดการเครื่องเสมือนพื้นฐาน (Create, Read, Update, Delete)
- เชื่อมต่อกับ Proxmox VE API

- พัฒนา Message Queue System
- สร้าง Dashboard หลักสำหรับผู้ใช้งาน

Phase 3: Advanced Features (สัปดาห์ที่ 9-12)

- พัฒนาระบบ Monitoring และ Resource Tracking
- เพิ่มระบบ Notification
- พัฒนา Advanced VM Management Features
- เพิ่มระบบ Template Management

Phase 4: Testing และ Optimization (สัปดาห์ที่ 13-16)

- Integration Testing และ Performance Testing
- Security Testing และ Load Testing
- UI/UX Improvements
- Documentation และ Deployment Preparation

บทที่ 4

ผลการดำเนินงาน

ในการพัฒนาระบบแพลตฟอร์มคลาวด์เพื่อสนับสนุนกิจกรรมการเรียนการสอนและงานวิชาการ ได้ดำเนินการออกแบบและพัฒนาระบบที่ประกอบด้วย 3 ส่วนหลัก คือ ส่วนติดต่อผู้ใช้ (Frontend) ส่วนบริการ API (API Service) และส่วนจัดการเครื่องเสมือน (VM Operations Service) พร้อมทั้งระบบฐานข้อมูล ระบบคิวข้อความ และระบบติดตามการทำงาน ในส่วนของผลการดำเนินงานที่ได้จะประกอบด้วยโครงสร้างโปรเจกต์ การจัดการไฟล์และโฟลเดอร์ การกำหนดค่าระบบ และการจัดการแพ็กเกจต่าง ๆ ทั้งนี้ ระบบที่พัฒนาขึ้นได้ถูกออกแบบให้รองรับการทำงานแบบ Microservices Architecture เพื่อความยืดหยุ่นในการขยายระบบและการบำรุงรักษา โดยใช้เทคโนโลยีสมัยใหม่ที่เหมาะสมกับการใช้งานในสภาพแวดล้อมการศึกษา

4.1 โครงสร้างโปรเจกต์หลัก

จากการออกแบบและพัฒนาระบบแพลตฟอร์มคลาวด์เพื่อสนับสนุนกิจกรรมการเรียนการสอนและงานวิชาการ ระบบประกอบไปด้วย 3 โฟลเดอร์หลัก ได้แก่

4.1.1 โฟลเดอร์ Apps

โฟลเดอร์ **Apps** เป็นที่เก็บแอปพลิเคชันหลักทั้งหมดของระบบ ซึ่งประกอบด้วย 3 บริการหลัก คือ **Midori**, **Momoi**, และ **Yuzu** ที่ทำงานร่วมกันในรูปแบบ Microservices Architecture

- 1) **Midori** - ส่วนติดต่อผู้ใช้ (Frontend) ที่พัฒนาด้วย Next.js
- 2) **Momoi** - บริการ API หลัก (API Service) ที่พัฒนาด้วย Elysia.js
- 3) **Yuzu** - บริการจัดการเครื่องเสมือน (VM Operations Service)

แต่ละบริการในโฟลเดอร์ **Apps** มีโครงสร้างและหน้าที่เฉพาะตัว โดยการแยกเป็นบริการย่อยจะช่วยให้ระบบมีความยืดหยุ่นในการพัฒนา การทดสอบ และการบำรุงรักษา

4.1.2 โฟลเดอร์ Packages

โฟลเดอร์ **Packages** ใช้สำหรับเก็บแพ็กเกจและไลบรารีที่ใช้ร่วมกันระหว่างบริการต่าง ๆ ในระบบ ภายในโฟลเดอร์นี้จะเก็บโค้ดที่สามารถนำไปใช้ซ้ำได้ (Reusable Code) เพื่อลดการเขียนโค้ดซ้ำซ้อน ตัวอย่างแพ็กเกจที่เก็บ ได้แก่

- 1) **Auth** - จัดการระบบยืนยันตัวตนและการให้สิทธิ์
- 2) **Database** - จัดการฐานข้อมูลและ ORM Models
- 3) **Utils** - ฟังก์ชันและเครื่องมือที่ใช้ร่วมกัน

การจัดระเบียบแพ็คเกจในโฟลเดอร์ **Packages** จะช่วยให้การจัดการโค้ดเป็นระเบียบ สะดวกต่อการบำรุงรักษา และสามารถนำไปใช้ในโปรเจกต์อื่น ๆ ได้ง่าย

4.1.3 โฟลเดอร์ Config

โฟลเดอร์ **Config** ใช้สำหรับเก็บไฟล์การกำหนดค่าต่าง ๆ ของระบบ เช่น การกำหนดค่าบริการ Infrastructure การตั้งค่า Monitoring และการกำหนดค่า Cloud Scripts ที่จำเป็นสำหรับการทำงานของระบบ โดยการแยกไฟล์การกำหนดค่าออกมาเป็นโฟลเดอร์แยกจะช่วยให้ง่ายต่อการจัดการและแก้ไขการตั้งค่าของระบบ โดยไม่ต้องไปแก้ไขโค้ดหลักของแอปพลิเคชัน

4.2 โครงสร้างแอปพลิเคชันและบริการ

4.2.1 ส่วนประกอบหลักของระบบ

นอกจากโฟลเดอร์ทั้งสามที่กล่าวมาแล้ว ในโครงสร้างโปรเจกต์ยังมีไฟล์และโฟลเดอร์สำคัญอื่น ๆ อีกหลายส่วน ได้แก่ ไฟล์ **package.json** ซึ่งเป็นไฟล์หลักที่ใช้ในการจัดการ Monorepo และกำหนด Workspaces สำหรับแต่ละแอปพลิเคชันและแพ็คเกจ ไฟล์นี้จะทำหน้าที่กำหนดการพึ่งพา (Dependencies) ระหว่างแพ็คเกจต่าง ๆ รวมถึงการตั้งค่า TypeScript และ Bun Runtime ที่ใช้ในการพัฒนาและรันแอปพลิเคชัน พร้อมทั้งจัดการ Scripts สำหรับการ Build, Test, และ Deploy ระบบ ทำให้ผู้พัฒนาสามารถจัดการโปรเจกต์ขนาดใหญ่ได้อย่างเป็นระบบและมีประสิทธิภาพ

4.2.2 โครงสร้างระบบทั้งหมด

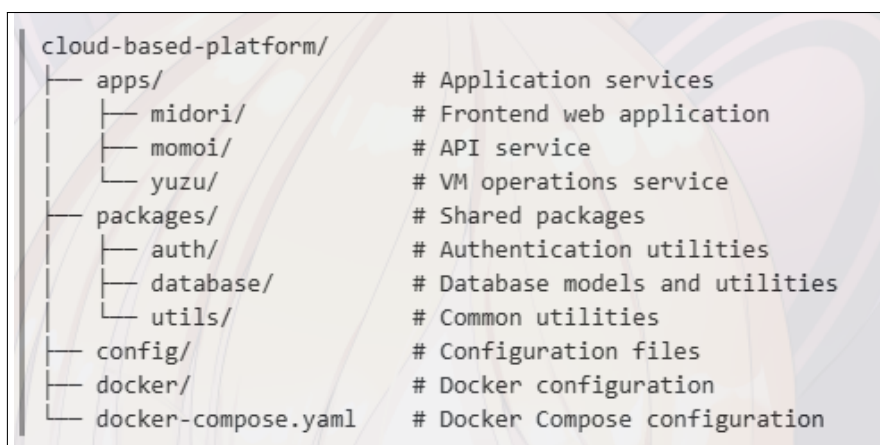
ภาพที่ 4-1 แสดงโครงสร้างโปรเจกต์ทั้งหมดของระบบแพลตฟอร์มคลาวด์เพื่อสนับสนุนกิจกรรมการเรียนการสอนและงานวิชาการ จะเห็นว่ามีการจัดระเบียบไฟล์และโฟลเดอร์อย่างเป็นระบบ เพื่อความสะดวกในการพัฒนาและบำรุงรักษา โดยโครงสร้างประกอบด้วยโฟลเดอร์หลัก 3 โฟลเดอร์ ได้แก่ Apps, Packages และ Config ซึ่งทำหน้าที่จัดการแอปพลิเคชัน แพ็คเกจรวม และการกำหนดค่าตามลำดับ อีกทั้งยังมีโฟลเดอร์เสริมอื่น ๆ เช่น Docker และ Docs ดังที่กล่าวไว้ข้างต้น

นอกจากนี้ยังประกอบด้วยไฟล์สำคัญอีกหลายไฟล์ ได้แก่

- docker-compose.yaml สำหรับการจัดการ Infrastructure Services
- tsconfig.json สำหรับการกำหนดค่า TypeScript ทั้งโปรเจกต์
- bun.lock สำหรับล็อกเวอร์ชันของแพ็คเกจที่ใช้

- .env.example สำหรับตัวอย่างการตั้งค่าตัวแปรสภาพแวดล้อม
- setup.sh และ setup.ps1 สำหรับสคริปต์การติดตั้งเริ่มต้น
- ไฟล์เอกสารต่าง ๆ เช่น README.md และ TODO.md

การแบ่งโครงสร้างเป็นส่วนย่อยเช่นนี้ช่วยให้การจัดการโปรเจกต์มีความเป็นระเบียบ และง่ายต่อการทำงานร่วมกันเป็นทีม เมื่อต้องการแก้ไขหรือเพิ่มฟีเจอร์ในบริการใดบริการหนึ่งก็สามารถเข้าไปทำงานได้โดยตรงในโฟลเดอร์นั้น ๆ โดยไม่กระทบกับส่วนอื่น ๆ ของระบบ



ภาพที่ 4-1 โครงสร้างโปรเจกต์ทั้งหมด

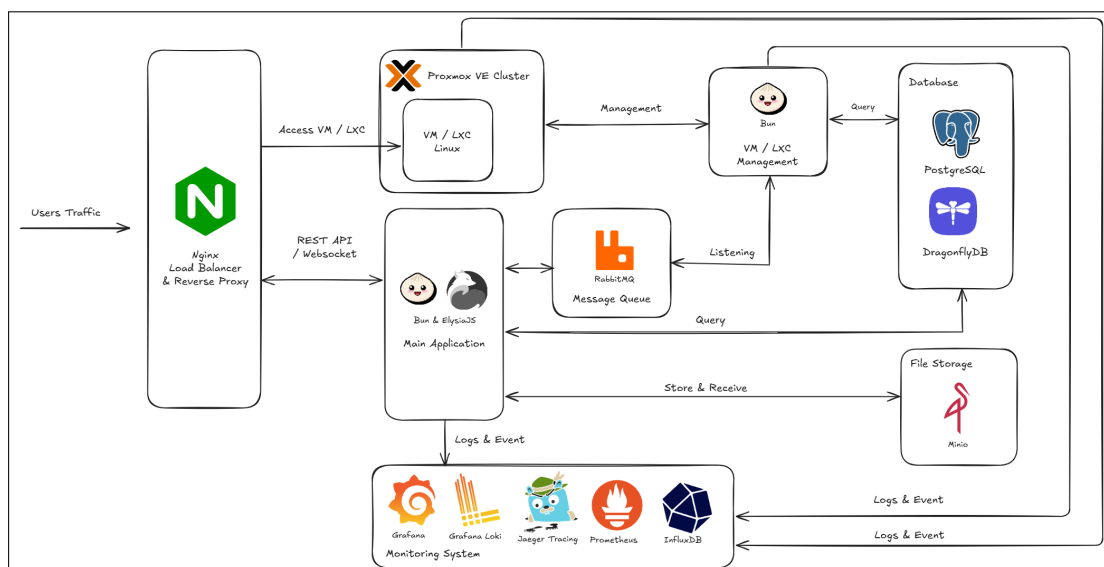
การจัดโครงสร้างโปรเจกต์ในลักษณะ Monorepo นี้ไม่เพียงแต่ช่วยให้โปรเจกต์เป็นระเบียบ แต่ยังช่วยลดความซับซ้อนในการจัดการ Dependencies ระหว่างแพ็คเกจ ทำให้การพัฒนาและการ Deploy เป็นทีมง่ายขึ้น และสามารถแชร์โค้ดระหว่างบริการต่าง ๆ ได้อย่างมีประสิทธิภาพ

4.3 สถาปัตยกรรมระบบและเทคโนโลยี

4.3.1 การออกแบบสถาปัตยกรรมแบบ Microservices

ระบบแพลตฟอร์มคลาวด์ที่พัฒนาขึ้นใช้สถาปัตยกรรมแบบ Microservices ซึ่งประกอบด้วยบริการหลัก 3 ตัว ที่ทำงานอิสระแต่สื่อสารกันผ่าน API และ Message Queue โดยแต่ละบริการมีหน้าที่และความรับผิดชอบที่แตกต่างกัน เพื่อให้ระบบมีความยืดหยุ่นและสามารถขยายตัวได้ง่าย

ภาพที่ 4-2 แสดงสถาปัตยกรรมระบบโดยรวม โดยมี **Midori** เป็นส่วนติดต่อผู้ใช้ที่พัฒนาด้วย Next.js และ Material UI **Momoi** เป็นบริการ API หลักที่จัดการการยืนยันตัวตน การจัดการผู้ใช้ และการเชื่อมต่อกับฐานข้อมูล และ **Yuzu** เป็นบริการจัดการเครื่องเสมือนที่เชื่อมต่อกับ Proxmox VE ผ่าน RabbitMQ



ภาพที่ 4-2 สถาปัตยกรรมระบบโดยรวม

4.3.2 เทคโนโลยีที่ใช้ในการพัฒนา

ในการพัฒนาระบบแพลตฟอร์มคลาวด์นี้ ได้เลือกใช้เทคโนโลยีที่ทันสมัยและเหมาะสมกับการใช้งานในสภาพแวดล้อมการศึกษา โดยเทคโนโลยีหลักที่ใช้ประกอบด้วย

- **Bun Runtime** - JavaScript/TypeScript Runtime ที่มีประสิทธิภาพสูง
- **Next.js** - React Framework สำหรับการพัฒนา Frontend
- **Elysia.js** - Web Framework สำหรับการพัฒนา API
- **PostgreSQL** - ระบบจัดการฐานข้อมูลเชิงสัมพันธ์
- **Prisma ORM** - Object-Relational Mapping สำหรับการจัดการฐานข้อมูล
- **RabbitMQ** - Message Queue สำหรับการสื่อสารระหว่างบริการ
- **Docker** - Container Platform สำหรับการ Deploy และการจัดการ Infrastructure

การเลือกใช้เทคโนโลยีเหล่านี้มีเหตุผลสำคัญ คือ ความเสถียร ประสิทธิภาพ และการรองรับการขยายระบบในอนาคต รวมถึงการมี Community ที่ใหญ่และเอกสารประกอบที่ครบถ้วน ทำให้การพัฒนาและบำรุงรักษาระบบเป็นไปอย่างราบรื่น

4.4 เอกสารประกอบการใช้งาน API และส่วนติดต่อผู้ใช้

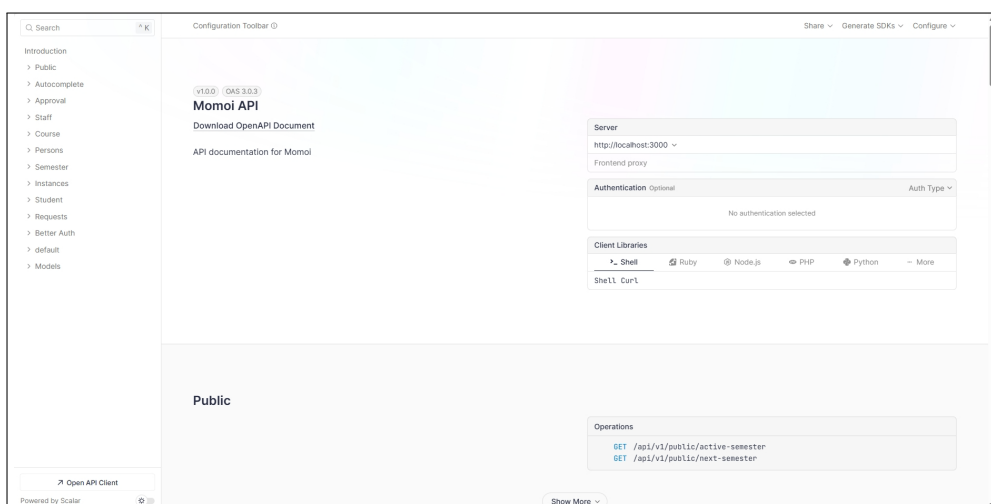
4.4.1 เอกสาร API Documentation

ระบบแพลตฟอร์มคลาวด์ได้จัดทำเอกสาร API Documentation อย่างครบถ้วนโดยใช้ OpenAPI Specification (Swagger) เพื่อให้ผู้พัฒนาและผู้ใช้งานสามารถเข้าใจและใช้งาน API ได้อย่างง่ายดาย

เอกสารนี้ประกอบด้วยรายละเอียดของ Endpoints ต่าง ๆ พารามิเตอร์ที่ต้องการ รูปแบบข้อมูลที่ส่งและรับ รวมทั้งตัวอย่างการใช้งานที่ชัดเจน

ภาพที่ 4-3 แสดงหน้าเอกสาร API Documentation ที่สร้างขึ้นโดยอัตโนมัติจากโค้ด TypeScript ผ่าน Elysia.js และ OpenAPI Plugin ซึ่งมีการจัดหมวดหมู่ API เป็นกลุ่มต่าง ๆ ดังนี้

- **Public API** - สำหรับการใช้งานทั่วไปที่ไม่ต้องมีการยืนยันตัวตน
- **Authentication API** - สำหรับการล็อกอิน ล็อกเอาต์ และการจัดการเซสชัน
- **Student API** - สำหรับนักศึกษาในการจัดการเครื่องเสมือนส่วนตัว
- **Staff API** - สำหรับอาจารย์และเจ้าหน้าที่ในการจัดการระบบ
- **Admin API** - สำหรับผู้ดูแลระบบในการจัดการผู้ใช้และทรัพยากร



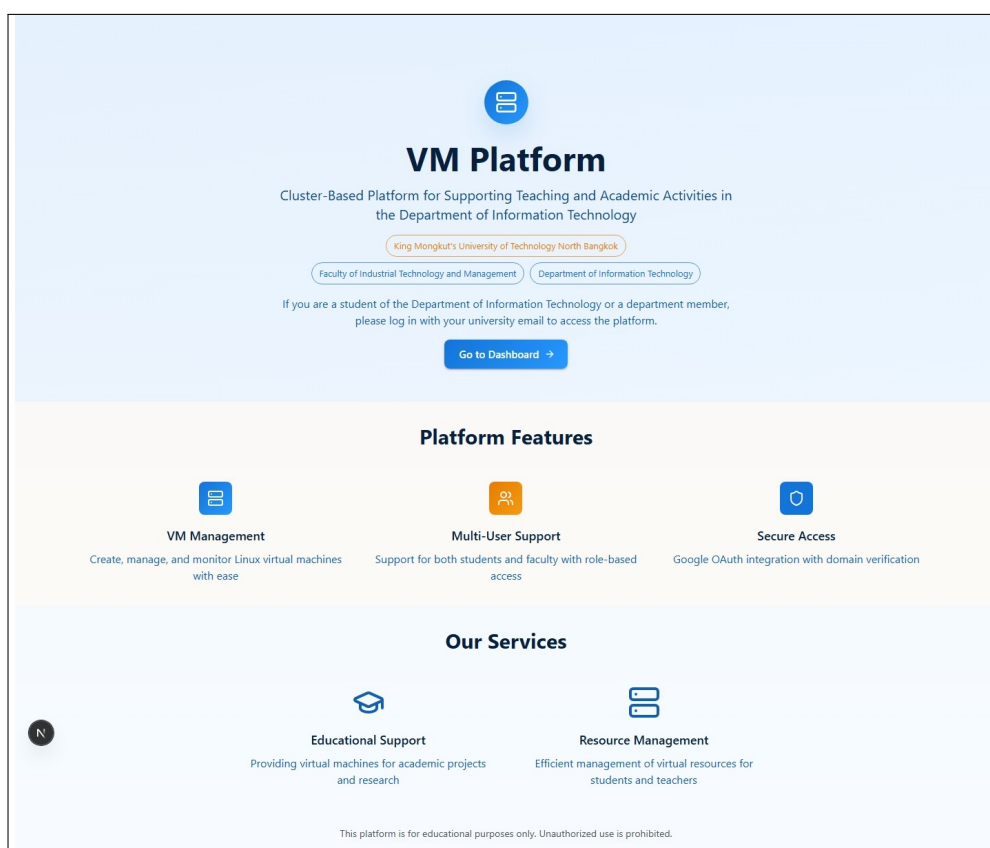
ภาพที่ 4-3 หน้าเอกสาร API Documentation

เอกสาร API นี้มีการอัปเดตอัตโนมัติเมื่อมีการเปลี่ยนแปลงโค้ด และสามารถทดสอบ API ได้โดยตรงผ่านหน้าเว็บ ทำให้ผู้พัฒนาสามารถทำความเข้าใจและใช้งาน API ได้อย่างรวดเร็วและถูกต้อง

4.4.2 ส่วนติดต่อผู้ใช้ (User Interface)

ส่วนติดต่อผู้ใช้ของระบบได้รับการออกแบบให้ใช้งานง่าย สวยงาม และตอบสนองได้ดีบนอุปกรณ์ต่าง ๆ โดยใช้ Material UI และ Tailwind CSS ในการจัดรูปแบบ ระบบประกอบด้วยหน้าหลักต่าง ๆ ที่มีการจัดระเบียบข้อมูลและฟังก์ชันการทำงานอย่างชัดเจน

ภาพที่ 4-4 แสดงหน้าแรกของระบบ (Landing Page) ที่ออกแบบให้ดึงดูดความสนใจและสื่อสารจุดประสงค์ของระบบได้ชัดเจน โดยมีการนำเสนอฟีเจอร์หลักของระบบ ข้อมูลการติดต่อ และลิงก์สำหรับการล็อกอินเข้าสู่ระบบ



ภาพที่ 4-4 หน้าแรกของระบบ (Landing Page)

4.4.3 Dashboard และระบบจัดการ

หลังจากที่ผู้ใช้ล็อกอินเข้าสู่ระบบแล้ว ผู้ใช้จะเข้าสู่หน้า Dashboard ที่แสดงข้อมูลสำคัญและฟังก์ชันการทำงานต่าง ๆ ตามบทบาทของผู้ใช้ โดย Dashboard ได้รับการออกแบบให้แสดงข้อมูลอย่างเป็นระบบและเข้าใจง่าย พร้อมทั้งมีการใช้งานที่สะดวกและรวดเร็ว

ส่วนติดต่อผู้ใช้ที่พัฒนาขึ้นมีลักษณะสำคัญดังนี้

- **Responsive Design** - รองรับการแสดงผลบนหน้าจอขนาดต่าง ๆ
- **Intuitive Navigation** - การนำทางที่เข้าใจง่ายและสอดคล้องกับมาตรฐาน UX
- **Real-time Updates** - การอัปเดตข้อมูลแบบเรียลไทม์
- **Accessibility** - การออกแบบที่คำนึงถึงผู้ใช้ที่มีความต้องการพิเศษ
- **Theme Support** - รองรับการเปลี่ยนธีมสีตามความต้องการ

การออกแบบส่วนติดต่อผู้ใช้ได้คำนึงถึงประสบการณ์ผู้ใช้ (User Experience) เป็นหลัก โดยมุ่งเน้นให้ผู้ใช้สามารถทำงานได้อย่างมีประสิทธิภาพและลดข้อผิดพลาดในการใช้งาน ทั้งนี้ระบบยังมีการจัดการข้อผิดพลาดและการแจ้งเตือนที่ชัดเจน เพื่อให้ผู้ใช้สามารถแก้ไขปัญหาได้อย่างรวดเร็ว

4.5 ตัวอย่างการใช้งานระบบตามบทบาทผู้ใช้

4.5.1 Dashboard สำหรับนักศึกษา

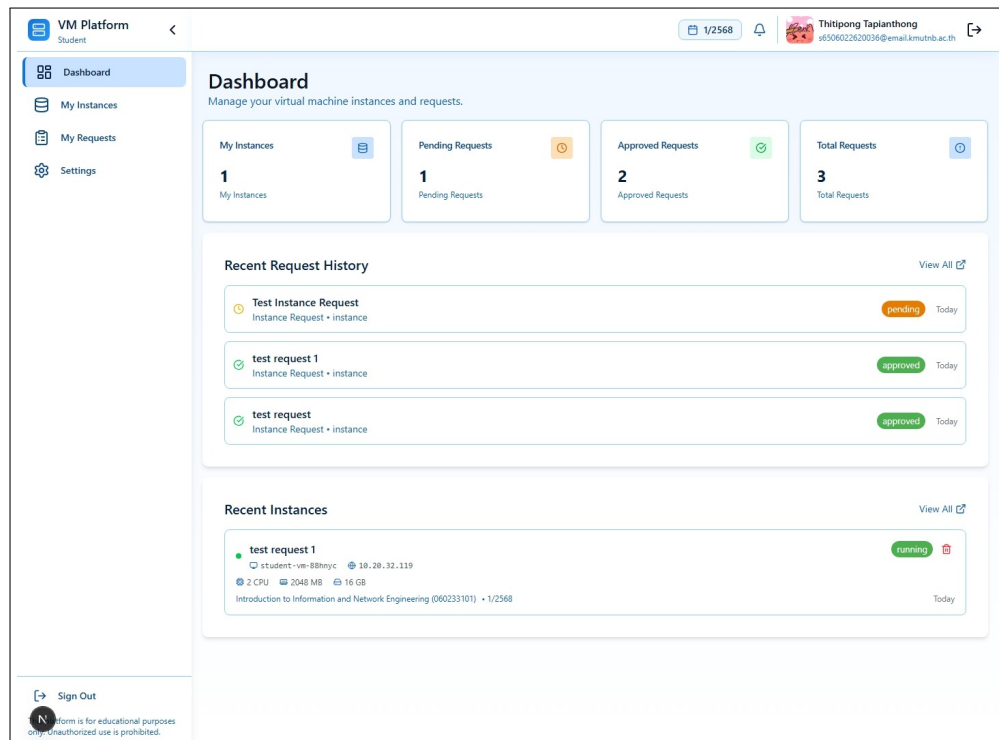
ระบบได้จัดทำ Dashboard เฉพาะสำหรับนักศึกษา เพื่อให้สามารถจัดการเครื่องเสมือนส่วนตัว และติดตามการใช้งานทรัพยากรได้อย่างง่ายดาย ภาพที่ 4-5 แสดง Dashboard หลักของนักศึกษาที่ประกอบด้วยข้อมูลสรุปการใช้งาน สถานะเครื่องเสมือน และฟังก์ชันการจัดการที่สำคัญ

นักศึกษาสามารถดูรายการเครื่องเสมือนทั้งหมดที่ตนเองมีสิทธิ์เข้าถึง พร้อมทั้งสถานะการทำงาน การใช้ทรัพยากร และการจัดการพื้นฐาน ดังแสดงในภาพที่ 4-6 ซึ่งนำเสนอรายละเอียดของแต่ละ Instance อย่างชัดเจน

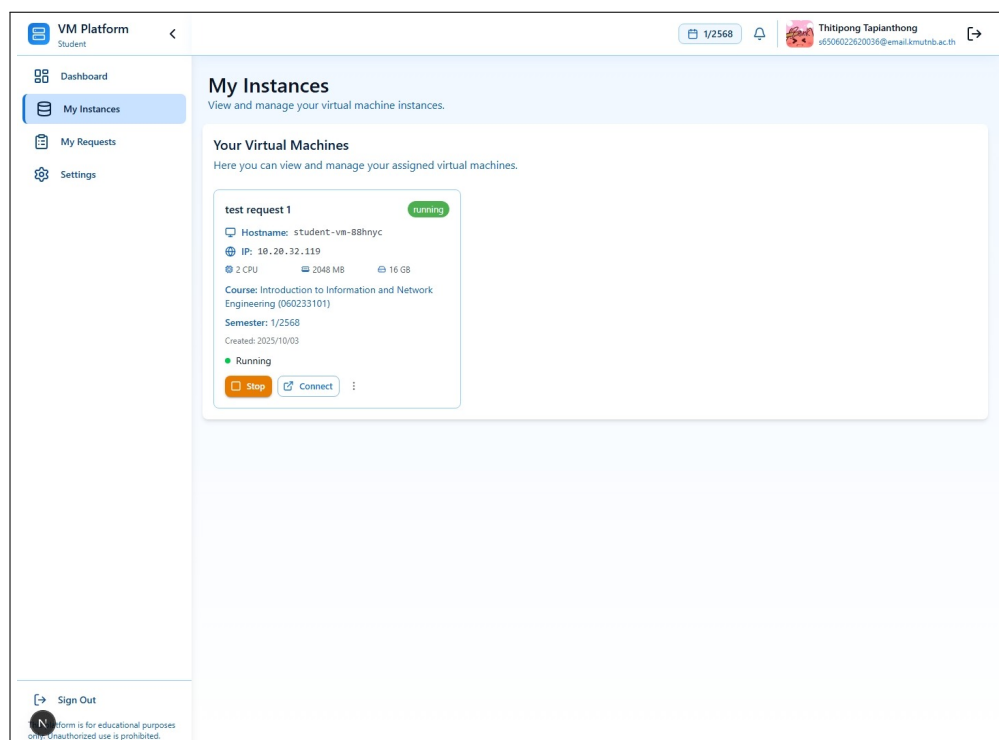
4.5.2 Dashboard สำหรับอาจารย์และเจ้าหน้าที่

สำหรับอาจารย์และเจ้าหน้าที่ ระบบได้จัดเตรียม Dashboard ที่มีฟังก์ชันการจัดการขั้นสูงมากขึ้น รวมถึงการอนุมัติคำขอ การจัดการหลักสูตร และการตรวจสอบการใช้งานของนักศึกษา ภาพที่ 4-7 แสดง Dashboard หลักสำหรับเจ้าหน้าที่ที่มีข้อมูลสถิติและการควบคุมระบบ

อาจารย์และเจ้าหน้าที่สามารถจัดการหลักสูตรและเทอมการศึกษา ดังแสดงในภาพที่ 4-8 ซึ่งช่วยให้สามารถกำหนดทรัพยากรและสิทธิ์การเข้าถึงสำหรับแต่ละวิชาได้อย่างเหมาะสม



ภาพที่ 4-5 Dashboard หลักสำหรับนักศึกษา



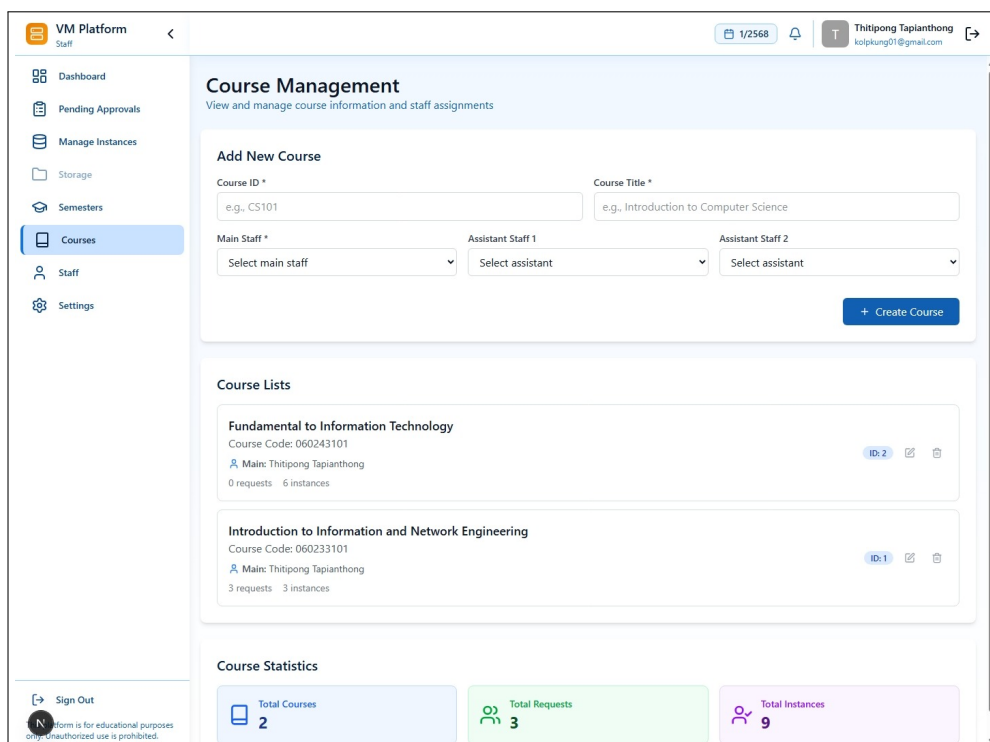
ภาพที่ 4-6 หน้าจัดการ VM Instances สำหรับนักศึกษา

The screenshot displays the VM Platform Dashboard for a staff member. The dashboard includes a sidebar with navigation options: Dashboard, Pending Approvals, Manage Instances, Storage, Semesters, Courses, Staff, and Settings. The main content area is divided into several sections:

- Dashboard Summary:** A row of four cards showing key metrics: Total Requests (1), Active Instances (5), Total Students (2), and Courses (2).
- Pending Requests:** A section titled "New virtual machine requests awaiting approval" showing a single request for a "Test Instance Request". The request details include: Hostname (student-vm-Z2qx47), 2 CPU, 2GB RAM, 16GB Storage, Requested by (Thitipong Tapianthong), Course (Introduction to Information and Network Engineering), and Requested date (2025/10/03). Action buttons for Edit, Reject, and Approve are available.
- Extension Requests:** A section titled "Requests to extend existing virtual machine instances" showing "No extension requests found".
- Recent Instances:** A section titled "Latest virtual machine instances across the platform" showing three instances: "more agent check", "test again agent check", and "test agent check". Each instance is shown with its name, status (running), user, creation date, IP, CPU, and RAM details. Action buttons for Archive and Stop are provided for each instance.

At the bottom left, there is a "Sign Out" button and a notice: "This platform is for educational purposes. Any unauthorized use is prohibited."

ภาพที่ 4-7 Dashboard หลักสำหรับอาจารย์และเจ้าหน้าที่



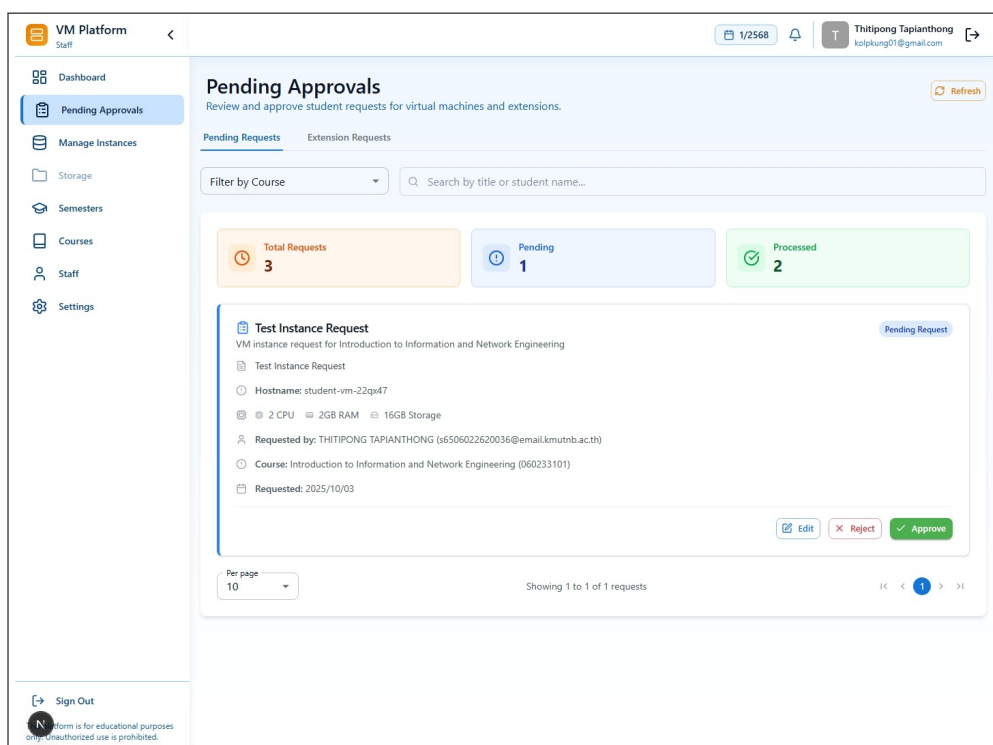
ภาพที่ 4-8 หน้าจัดการหลักสูตรและรายวิชา

นอกจากนี้ ระบบยังมีหน้าสำหรับการอนุมัติคำขอต่าง ๆ จากนักศึกษา ดังแสดงในภาพที่ 4-9 ซึ่งช่วยให้อาจารย์สามารถตรวจสอบและอนุมัติคำขอได้อย่างมีประสิทธิภาพ

4.5.3 ระบบจัดการแบบ Role-Based Access Control

ระบบได้นำเอา Role-Based Access Control (RBAC) มาใช้ในการจัดการสิทธิ์การเข้าถึงข้อมูล และฟังก์ชันต่าง ๆ โดยแบ่งผู้ใช้เป็น 3 กลุ่มหลัก คือ นักศึกษา อาจารย์ และผู้ดูแลระบบ ซึ่งแต่ละกลุ่มจะมีสิทธิ์และหน้าที่ที่แตกต่างกันตามความเหมาะสม

การ ออกแบบ ระบบใน ลักษณะ นี้ช่วยให้การจัดการ ความปลอดภัย ของข้อมูล เป็นไปอย่างมีประสิทธิภาพ และผู้ใช้แต่ละกลุ่มจะเห็นเฉพาะข้อมูลและฟังก์ชันที่เกี่ยวข้องกับหน้าที่ของตนเอง ทำให้การใช้งานง่ายขึ้นและลดความเสี่ยงในการเข้าถึงข้อมูลที่ไม่เหมาะสม



ภาพที่ 4-9 หน้าอนุมัติคำขอที่รอดำเนินการ

บทที่ 5

สรุปผลการดำเนินงาน

5.1 สรุปผลการดำเนินงาน

โครงการ "Cloud-Based Platform for Supporting Teaching and Academic Activities" ได้พัฒนาแพลตฟอร์มคลาวด์สำหรับสนับสนุนการเรียนการสอนและกิจกรรมทางวิชาการในภาควิชาเทคโนโลยีสารสนเทศ โดยใช้สถาปัตยกรรมแบบ Microservices ที่ประกอบด้วย 3 ส่วนหลัก ได้แก่ Midori (Frontend), Momoi (API Service), และ Yuzu (VM Operations) ผลลัพธ์ที่สำคัญของโครงการคือการสร้างระบบจัดการเครื่องเสมือน (Virtual Machine) ที่สมบูรณ์ ซึ่งสามารถสร้าง กำหนดค่า เริ่มต้น หยุด และลบเครื่องเสมือนได้ตามความต้องการ พร้อมระบบควบคุมการเข้าถึงแบบ Role-Based Access Control (RBAC) สำหรับนักศึกษา อาจารย์ และผู้ดูแลระบบ ทำให้สามารถใช้ทรัพยากรเซิร์ฟเวอร์ภายในภาควิชาได้อย่างมีประสิทธิภาพ และช่วยเพิ่มความสะดวกในการจัดการสภาพแวดล้อมการเรียนรู้แบบดิจิทัลให้กับอาจารย์และนักศึกษา

5.2 ปัญหาและอุปสรรค

- 5.2.1 ความซับซ้อนของการจัดการ Microservices Architecture เนื่องจากระบบประกอบด้วยหลายส่วนที่ต้องทำงานร่วมกันผ่าน API และ Message Queue ทำให้การดีบั๊กและแก้ไขปัญหาเป็นเรื่องที่ท้าทาย
- 5.2.2 การจัดการ Virtual Machine ผ่าน Proxmox VE API ที่มีความซับซ้อนในการสื่อสารและจัดการ State ของ VM รวมถึงการจัดสรร IP Address และ Resource อย่างอัตโนมัติ
- 5.2.3 การออกแบบระบบ Role-Based Access Control (RBAC) ที่ปลอดภัยและครอบคลุมผู้ใช้งานทุกระดับ ตั้งแต่ นักศึกษา อาจารย์ และผู้ดูแลระบบ โดยต้องคำนึงถึงความปลอดภัยและการจำกัดสิทธิ์การเข้าถึง
- 5.2.4 การใช้เทคโนโลยีใหม่ เช่น Bun Runtime, Elysia.js Framework และ RabbitMQ Message Queue ที่ต้องใช้เวลาในการศึกษาและทำความเข้าใจการทำงาน
- 5.2.5 การจัดการ Monitoring และ Observability ด้วย OpenTelemetry, Jaeger, Prometheus และ Grafana เพื่อติดตามประสิทธิภาพของระบบ

5.3 การแก้ปัญหา

- 5.3.1 ใช้ Docker และ Docker Compose ในการจัดการ Microservices เพื่อให้แต่ละ Service สามารถทำงานแยกกันได้อย่างเป็นอิสระ และจัดทำ API Documentation ด้วย OpenAPI เพื่อให้การสื่อสารระหว่าง Service เป็นมาตรฐาน
- 5.3.2 พัฒนา Yuzu Service ที่ทำหน้าที่เป็น Wrapper สำหรับ Proxmox VE API โดยใช้ RabbitMQ เป็น Message Queue เพื่อจัดการ VM Operations แบบ Asynchronous และสร้างระบบจัดการ IP Pool อัตโนมัติ
- 5.3.3 ออกแบบ Database Schema ด้วย Prisma ORM เพื่อจัดการข้อมูลผู้ใช้และสิทธิ์การเข้าถึง และใช้ JWT Token สำหรับ Authentication พร้อมกำหนด Role และ Permission ที่ชัดเจน
- 5.3.4 จัดทำ Documentation และ Example Code ที่ครอบคลุม พร้อมใช้ TypeScript เป็นหลักในการพัฒนาเพื่อให้ได้ Type Safety และลดข้อผิดพลาดในการพัฒนา
- 5.3.5 ใช้ OpenTelemetry เป็น Standard สำหรับ Observability และติดตั้ง Monitoring Stack ประกอบด้วย Jaeger สำหรับ Distributed Tracing, Prometheus สำหรับ Metrics Collection และ Grafana สำหรับ Visualization

5.4 ข้อเสนอแนะ

- 5.4.1 พัฒนาระบบ Auto-scaling และ Load Balancing เพื่อรองรับจำนวนผู้ใช้งานที่เพิ่มขึ้น และเพิ่มความสามารถในการจัดการ Resource Allocation ให้มีประสิทธิภาพมากขึ้น
- 5.4.2 เพิ่มฟีเจอร์ VM Template Management เพื่อให้อาจารย์สามารถสร้างและจัดการ Template สำหรับรายวิชาต่างๆ ได้ตามความต้องการ และเพิ่มระบบ Backup และ Snapshot Management
- 5.4.3 พัฒนา Mobile Application หรือ Progressive Web App (PWA) เพื่อให้สามารถเข้าถึงระบบได้จากอุปกรณ์มือถือ และเพิ่มระบบ Notification เพื่อแจ้งเตือนสถานะของ VM
- 5.4.4 เพิ่มระบบ CI/CD Pipeline สำหรับการ Deploy และ Testing อัตโนมัติ และจัดทำ Comprehensive Testing Suite ประกอบด้วย Unit Tests, Integration Tests และ End-to-End Tests
- 5.4.5 ศึกษาการใช้ Kubernetes สำหรับการจัดการ Container Orchestration เพื่อเพิ่มความยืดหยุ่นและ Scalability

บรรณานุกรม

- Terry Anderson. (2008). The theory and practice of online learning (2nd).
- Jonathan ChyaXunfei Jiang. (2024). Building a cloud infrastructure for virtual machine scheduling in datacenters [Accessed 25 June 2025.]. *Proceedings of IEEE Conference*. Retrieved June 25, 2025, from <https://ieeexplore.ieee.org/document/10427797>
- Richard C. ClarkAnn Kwinn. (2016). The new virtual classroom: Evidence-based guidelines for synchronous e-learning (2nd).
- Carlos GarciaMaria Rodriguez. (2023). Performance evaluation of hypervisor technologies in educational environments: Vmware vsphere vs. proxmox ve vs. kvm. *Journal of Network and Computer Applications*, 203, 103–121. <https://doi.org/10.1016/j.jnca.2022.103401>
- Shahidullah KaiserMd. SadunhaqAli Aman TosunTurgay Korkmaz. (2022). Container technologies for arm architecture: A comprehensive survey of the state-of-the-art [Accessed 23 June 2025.]. *IEEE Communications Surveys Tutorials*. Retrieved June 23, 2025, from <https://ieeexplore.ieee.org/document/9852232>
- Paul Hoffman Karen Scarfone. (2011). *Guide to security for full virtualization technologies*. Retrieved June 23, 2025, from <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-125.pdf>
- David LeeWei Chen. (2021). Container technology in software development education: A comparative study of docker and kubernetes. *IEEE Transactions on Education*, 64(2), 187–194. <https://doi.org/10.1109/TE.2020.3015678>
- Naresh NayakDennis GreweSebastian Schildt. (2023). Automotive container orchestration: Requirements, challenges and open directions [Accessed 23 June 2025.]. *Proceedings of IEEE Conference*. Retrieved June 23, 2025, from <https://ieeexplore.ieee.org/document/10136278>
- Rawiporn SuamsiriKamonwan Janmanee. (2024). Progress tracking system for special project documents [Accessed 23 June 2025.]. Retrieved June 23, 2025, from <http://202.44.47.109/project2/2567-1/671015-1.pdf>

บรรณานุกรม (ต่อ)

- Danairat T. (2015). *Cloud computing technology the architecture overview*. Retrieved June 23, 2025, from https://www.dga.or.th/wp-content/uploads/2015/05/file_c08274b0a8d4c29561b7e7314afaa102.pdf
- Yu TaoGang Chen. (2016). An extensible universal reverse proxy architecture [Accessed 23 June 2025.]. *Proceedings of IEEE Conference*. Retrieved June 23, 2025, from <https://ieeexplore.ieee.org/document/7945939>

ภาคผนวก

ภาคผนวก ก.

การติดตั้งและใช้งาน Cloud-Based Platform

ก.1 การติดตั้งและใช้งาน Cloud-Based Platform

ในการติดตั้งและใช้งาน Cloud-Based Platform สำหรับการสนับสนุนกิจกรรมการเรียนการสอนและกิจกรรมทางวิชาการ ผู้ใช้งานจำเป็นต้องเตรียมความพร้อมของระบบและทำการติดตั้งตามขั้นตอนดังนี้

ข้อกำหนดของระบบ (System Requirements)

- 1) ซอฟต์แวร์ที่จำเป็น:
 - 1.1) Bun 1.2.0 หรือสูงกว่า (<https://bun.sh/>)
 - 1.2) Docker และ Docker Compose (<https://www.docker.com/>)
 - 1.3) PostgreSQL 15 หรือสูงกว่า
 - 1.4) RabbitMQ 3.12 หรือสูงกว่า
 - 1.5) Proxmox VE (สำหรับการใช้งานจริง)
- 2) ข้อกำหนดของฮาร์ดแวร์:
 - 2.1) RAM: อย่างน้อย 8 GB (แนะนำ 16 GB หรือมากกว่า)
 - 2.2) CPU: อย่างน้อย 4 cores
 - 2.3) พื้นที่เก็บข้อมูล: อย่างน้อย 50 GB
 - 2.4) เชื่อมต่ออินเทอร์เน็ตเสถียร

การดาวน์โหลดและติดตั้งโปรเจกต์

- 1) เปิด Terminal หรือ Command Prompt และทำการโคลนโปรเจกต์จาก GitHub Repository

```
git clone https://github.com/akikungz/cloud-based-platform.git
cd cloud-based-platform
```

ภาพที่ ก-1 คำสั่งสำหรับโคลนโปรเจกต์

- 2) ติดตั้ง Dependencies ของโปรเจกต์

```
bun install
```

ภาพที่ ก-2 คำสั่งสำหรับติดตั้ง Dependencies

- 3) ตั้งค่า Environment Variables โดยคัดลอกไฟล์ตัวอย่าง

ก.2 การเริ่มต้นใช้งานระบบ

หลังจากติดตั้งโปรเจกต์เรียบร้อยแล้ว สามารถเริ่มต้นใช้งานระบบได้ดังนี้

```
cp .env.example .env
# .env
```

ภาพที่ ก-3 การตั้งค่า Environment Variables

การรันระบบในโหมด Development

- 1) เริ่มต้น Infrastructure Services ด้วย Docker Compose

```
docker-compose up -d
```

ภาพที่ ก-4 การเริ่มต้น Infrastructure Services

- 2) เริ่มต้น Midori (Frontend Application)

```
cd apps/midori
bun dev
```

ภาพที่ ก-5 การเริ่มต้น Midori Frontend

- 3) เริ่มต้น Momoi (API Service) ใน Terminal ใหม่
- 4) เริ่มต้น Yuzu (VM Operations Service) ใน Terminal ใหม่
- 5) เข้าใช้งานระบบผ่าน Web Browser ที่ <http://localhost:3000>

ก.3 การใช้งานส่วนประกอบต่างๆ ของระบบ

ระบบ Cloud-Based Platform ประกอบด้วยส่วนประกอบหลัก 3 ส่วน ดังนี้

Midori - Frontend Application

- 1) เป็น Web Application ที่พัฒนาด้วย Next.js 15.5
- 2) ใช้ Material UI 7.3 และ Tailwind CSS 4 สำหรับส่วน User Interface
- 3) รองรับการใช้งานบนอุปกรณ์ Desktop และ Mobile
- 4) มีฟีเจอร์หลัก ดังนี้:
 - 4.1) การจัดการ Virtual Machines (สร้าง, ติดตาม, จัดการ)
 - 4.2) Dashboard สำหรับดูการใช้งานทรัพยากร
 - 4.3) ระบบ Authentication และการจัดการบัญชีผู้ใช้
 - 4.4) รองรับการใช้งานแบบ Responsive Design
- 5) URL สำหรับการเข้าใช้งาน: <http://localhost:3000>

Momoi - API Service

```
cd apps/momoi
bun dev
```

ภาพที่ ก-6 การเริ่มต้น Momoi API Service

```
cd apps/yuzu
bun dev
```

ภาพที่ ก-7 การเริ่มต้น Yuzu VM Operations Service

- 1) เป็น Core API Service ที่พัฒนาด้วย Elysia.js
- 2) ทำหน้าที่เป็นตัวกลางระหว่าง Frontend และ VM Operations Service
- 3) มีฟีเจอร์หลัก ดังนี้:
 - 3.1) RESTful API สำหรับการทำงานของแพลตฟอร์ม
 - 3.2) OpenAPI Documentation แบบอัตโนมัติ
 - 3.3) การติดตาม Telemetry ด้วย Jaeger และ Prometheus
 - 3.4) การเชื่อมต่อกับ RabbitMQ สำหรับการประมวลผลแบบ Asynchronous
- 4) URL สำหรับ API: <http://localhost:3001>

Yuzu - VM Operations Service

- 1) เป็น Service สำหรับจัดการ Virtual Machines ใน Proxmox VE
- 2) ประมวลผลคำขอผ่าน RabbitMQ Message Queue
- 3) มีฟีเจอร์หลัก ดังนี้:
 - 3.1) จัดการ VM Lifecycle (สร้าง, เริ่ม, หยุด, ลบ VMs)
 - 3.2) การ Provisioning แบบ Template-based
 - 3.3) การจัดการ IP Address แบบอัตโนมัติ
 - 3.4) ระบบ Logging ที่ครอบคลุมด้วย Pino

ก.4 การตรวจสอบและติดตามระบบ

ระบบมี Services สำหรับการตรวจสอบและติดตามการทำงานดังนี้

Monitoring Services

- 1) **Prometheus** - การเก็บข้อมูล Metrics
 - 1.1) URL: <http://localhost:9090>
 - 1.2) ใช้สำหรับติดตามประสิทธิภาพของระบบ
- 2) **Jaeger** - การติดตาม Distributed Tracing

- 2.1) URL: <http://localhost:16686>
- 2.2) ใช้สำหรับติดตามการทำงานของ API calls
- 3) **Grafana** - Dashboard สำหรับการแสดงผล
 - 3.1) URL: <http://localhost:3002>
 - 3.2) Username/Password: admin/admin (ค่าเริ่มต้น)
 - 3.3) ใช้สำหรับแสดงข้อมูลสถิติและ Metrics
- 4) **RabbitMQ Management** - การจัดการ Message Queue
 - 4.1) URL: <http://localhost:15672>
 - 4.2) Username/Password: user/password (ค่าเริ่มต้น)
 - 4.3) ใช้สำหรับตรวจสอบ Message Queue

ก.5 การแก้ไขปัญหาเบื้องต้น

หากพบปัญหาในการใช้งาน สามารถแก้ไขได้ดังนี้

ปัญหาที่พบบ่อยและวิธีแก้ไข

- 1) **Port ถูกใช้งานแล้ว**
 - 1.1) ตรวจสอบ Port ที่ใช้งานด้วยคำสั่ง: `lsof -i :3000` (สำหรับ macOS/Linux)
 - 1.2) หยุดกระบวนการที่ใช้ Port หรือเปลี่ยน Port ในไฟล์ `.env`
- 2) **Docker Services ไม่สามารถเริ่มได้**
 - 2.1) ตรวจสอบสถานะด้วยคำสั่ง: `docker-compose ps`
 - 2.2) ดู Log ด้วยคำสั่ง: `docker-compose logs [service_name]`
 - 2.3) รีสตาร์ท Services ด้วยคำสั่ง: `docker-compose restart`
- 3) **Database Connection Error**
 - 3.1) ตรวจสอบว่า PostgreSQL Container ทำงานอยู่
 - 3.2) ตรวจสอบการตั้งค่าใน Environment Variables
 - 3.3) รันคำสั่ง Database Migration (หากจำเป็น)
- 4) **Frontend ไม่สามารถเชื่อมต่อ API ได้**
 - 4.1) ตรวจสอบว่า Momoi API Service ทำงานอยู่ที่ Port 3001
 - 4.2) ตรวจสอบการตั้งค่า API URL ในไฟล์ Environment
 - 4.3) ตรวจสอบ CORS Settings ใน API Service

การเข้าถึงข้อมูลเพิ่มเติม

- 1) อ่าน Documentation เพิ่มเติมใน **README.md** ของแต่ละ Component
- 2) ตรวจสอบ Log Files สำหรับข้อมูลการ Debug
- 3) ใช้ Jaeger UI สำหรับติดตามการทำงานของ API
- 4) ใช้ Grafana Dashboard สำหรับติดตามประสิทธิภาพระบบ

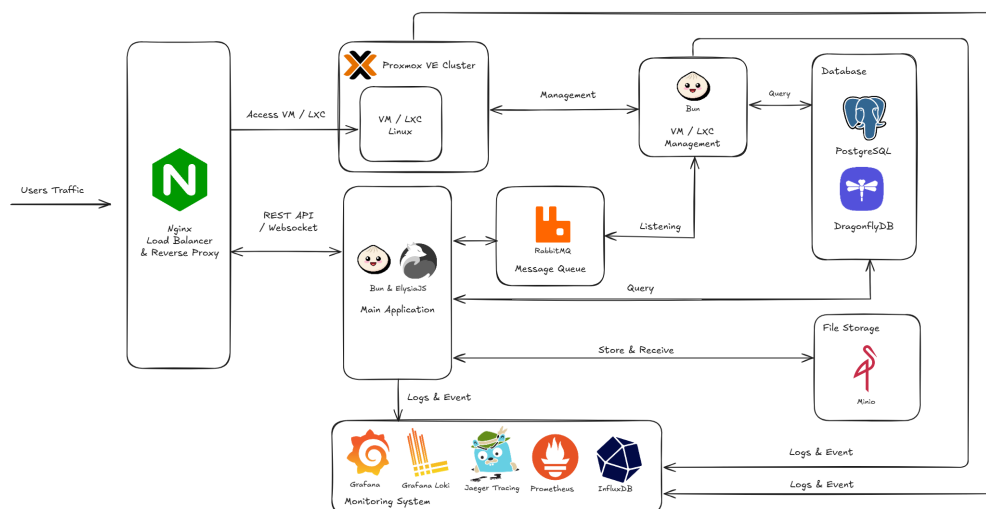
ภาคผนวก ข.

สถาปัตยกรรมระบบและส่วนประกอบทางเทคนิค

ข.1 สถาปัตยกรรมระบบและส่วนประกอบทางเทคนิค

ระบบ Cloud-Based Platform ได้รับการออกแบบด้วยสถาปัตยกรรมแบบ Microservices เพื่อให้การพัฒนา การดูแลรักษา และการขยายระบบเป็นไปอย่างมีประสิทธิภาพ โดยมีรายละเอียดส่วนประกอบต่างๆ ดังนี้

ภาพรวมสถาปัตยกรรมระบบ



ภาพที่ ข-1 สถาปัตยกรรมระบบ Cloud-Based Platform

ข.2 เทคโนโลยีที่ใช้ในการพัฒนา

ระบบได้รับการพัฒนาด้วยเทคโนโลยีสมัยใหม่เพื่อประสิทธิภาพและความเสถียรสูงสุด

Frontend Technologies

- 1) **Next.js 15.5** - React Framework สำหรับการพัฒนา Web Application
 - 1.1) รองรับ Server-Side Rendering (SSR) และ Static Site Generation (SSG)
 - 1.2) มี Built-in Performance Optimization
 - 1.3) รองรับ API Routes สำหรับ Backend Integration
- 2) **Material UI 7.3** - Component Library สำหรับ User Interface
 - 2.1) มี Pre-built Components ที่สวยงามและใช้งานง่าย
 - 2.2) รองรับ Theming และ Customization
 - 2.3) มี Accessibility Support ในตัว
- 3) **Tailwind CSS 4** - Utility-first CSS Framework
 - 3.1) ให้ความยืดหยุ่นสูงในการปรับแต่ง Styling

- 3.2) มีขนาดเล็กและประสิทธิภาพสูง
- 3.3) รองรับ Responsive Design ได้ดี
- 4) **Better Auth** - Authentication Library
 - 4.1) รองรับการ Authentication หลากหลายรูปแบบ
 - 4.2) มีความปลอดภัยสูง
 - 4.3) ง่ายต่อการ Integration

Backend Technologies

- 1) **Elysia.js** - Web Framework สำหรับ API Development
 - 1.1) ประสิทธิภาพสูงด้วย Bun Runtime
 - 1.2) รองรับ TypeScript แบบ Native
 - 1.3) มี Built-in OpenAPI Documentation
 - 1.4) รองรับ Plugin System ที่ยืดหยุ่น
- 2) **Bun** - JavaScript Runtime และ Package Manager
 - 2.1) เร็วกว่า Node.js อย่างมีนัยสำคัญ
 - 2.2) รองรับ TypeScript โดยไม่ต้องติดตั้งเพิ่มเติม
 - 2.3) มี Built-in Package Manager, Bundler, และ Test Runner
- 3) **PostgreSQL** - Relational Database Management System
 - 3.1) ความเสถียรและประสิทธิภาพสูง
 - 3.2) รองรับ Advanced SQL Features
 - 3.3) มี ACID Compliance
- 4) **Prisma ORM** - Database Access Layer
 - 4.1) Type-safe Database Access
 - 4.2) Auto-generated Database Client
 - 4.3) Database Migration Management

Message Queue และ Communication

- 1) **RabbitMQ** - Message Broker สำหรับ Asynchronous Communication
 - 1.1) รองรับ Message Patterns หลากหลายรูปแบบ
 - 1.2) มีความเสถียรสูงและ Message Durability
 - 1.3) มี Management Interface สำหรับการติดตาม

2) **AMQP (Advanced Message Queuing Protocol)**

- 2.1) Protocol มาตรฐานสำหรับ Message Queuing
- 2.2) รองรับการส่ง Message แบบ Reliable
- 2.3) มี Message Routing ที่ยืดหยุ่น

ข.3 Monitoring และ Observability

ระบบมีการติดตามและตรวจสอบการทำงานอย่างครอบคลุม

Metrics Collection

- 1) **Prometheus** - Time Series Database สำหรับ Metrics
 - 1.1) เก็บข้อมูล Performance Metrics
 - 1.2) รองรับ PromQL สำหรับ Query ข้อมูล
 - 1.3) มี Alerting System ในตัว
- 2) **Elysia Prometheus Plugin** - Metrics Integration
 - 2.1) ติดตาม HTTP Request/Response Metrics
 - 2.2) วัดประสิทธิภาพของ API Endpoints
 - 2.3) รองรับ Custom Metrics

Distributed Tracing

- 1) **Jaeger** - Distributed Tracing System
 - 1.1) ติดตามการเดินทางของ Request ข้ามหลาย Services
 - 1.2) วิเคราะห์ Performance Bottlenecks
 - 1.3) มี UI สำหรับการแสดงผล Traces
- 2) **OpenTelemetry** - Observability Framework
 - 2.1) มาตรฐานสำหรับ Telemetry Data Collection
 - 2.2) รองรับ Metrics, Traces, และ Logs
 - 2.3) มี Vendor-neutral Architecture

Logging และ Log Aggregation

- 1) **Pino** - High Performance Logging Library
 - 1.1) JSON-based Structured Logging
 - 1.2) ประสิทธิภาพสูงมาก
 - 1.3) รองรับ Log Levels และ Child Loggers

- 2) **Loki** - Log Aggregation System
 - 2.1) เก็บรวบรวม Logs จากหลาย Sources
 - 2.2) ใช้ Label-based Indexing
 - 2.3) Integration ที่ดีกับ Grafana
- 3) **Grafana** - Visualization Platform
 - 3.1) สร้าง Dashboard สำหรับ Metrics และ Logs
 - 3.2) รองรับ Multiple Data Sources
 - 3.3) มี Alerting และ Notification Features

ข.4 Virtualization และ Infrastructure

ระบบใช้เทคโนโลยี Virtualization สำหรับการจัดการ Virtual Machines

Hypervisor Technology

- 1) **Proxmox VE** - Enterprise Virtualization Platform
 - 1.1) รองรับทั้ง KVM Virtual Machines และ LXC Containers
 - 1.2) มี Web-based Management Interface
 - 1.3) รองรับ High Availability และ Clustering
 - 1.4) มี Backup และ Restore Features
- 2) **KVM (Kernel-based Virtual Machine)**
 - 2.1) Hypervisor แบบ Type 1 ที่มีประสิทธิภาพสูง
 - 2.2) รองรับ Hardware Virtualization
 - 2.3) มีความเสถียรและความปลอดภัยสูง

Container Technology

- 1) **Docker** - Container Platform
 - 1.1) ใช้สำหรับ Package Applications และ Dependencies
 - 1.2) รองรับการ Deploy แบบ Consistent
 - 1.3) มีความเร็วในการ Start/Stop
- 2) **Docker Compose** - Multi-container Application Definition
 - 2.1) จัดการ Multiple Containers ด้วยไฟล์ YAML
 - 2.2) รองรับ Service Dependencies
 - 2.3) ง่ายต่อการ Setup Development Environment

ข.5 Data Validation และ Type Safety

ระบบใช้เทคโนโลยีสำหรับการตรวจสอบข้อมูลและความปลอดภัยของประเภทข้อมูล

Schema Validation

- 1) **Zod** - TypeScript Schema Validation Library
 - 1.1) Type-safe Schema Definition
 - 1.2) Runtime Validation
 - 1.3) Auto-generated TypeScript Types
 - 1.4) รองรับ Complex Data Structures
- 2) **TypeScript** - Static Type Checking
 - 2.1) Compile-time Type Safety
 - 2.2) Enhanced IDE Support
 - 2.3) Better Code Quality และ Maintainability

ข.6 ข้อดีของสถาปัตยกรรมที่เลือกใช้

การเลือกใช้สถาปัตยกรรมและเทคโนโลยีดังกล่าวมีข้อดีดังนี้

ประสิทธิภาพและความเร็ว

- 1) Bun Runtime มีประสิทธิภาพสูงกว่า Node.js อย่างมีนัยสำคัญ
- 2) Elysia.js ให้ Throughput สูงและ Latency ต่ำ
- 3) PostgreSQL มีประสิทธิภาพดีในการจัดการข้อมูลขนาดใหญ่
- 4) Microservices Architecture ช่วยให้ Scale แต่ละส่วนได้อย่างอิสระ

ความยืดหยุ่นและการขยายระบบ

- 1) แต่ละ Service สามารถ Deploy และ Update แยกกันได้
- 2) RabbitMQ ช่วยให้ระบบทำงานแบบ Asynchronous ได้
- 3) Docker ทำให้การ Deploy ง่ายและ Consistent
- 4) Prometheus และ Grafana ให้ Observability ที่ดี

ความปลอดภัยและความเสถียร

- 1) TypeScript และ Zod ให้ Type Safety ที่แข็งแกร่ง
- 2) Better Auth ให้ระบบ Authentication ที่ปลอดภัย

- 3) PostgreSQL มี ACID Compliance
- 4) Microservices ช่วยลด Impact ของ Single Point of Failure

ภาคผนวก ค.
การ Deploy และการตั้งค่าสำหรับ Production

ค.1 การ Deploy และการตั้งค่าสำหรับ Production

การนำระบบ Cloud-Based Platform ไปใช้งานจริง (Production) ต้องมีการตั้งค่าและการเตรียมการที่แตกต่างจากการใช้งานในโหมด Development โดยมีรายละเอียดดังนี้

การเตรียมสภาพแวดล้อม Production

1) ข้อกำหนดของ Server:

- 1.1) CPU: อย่างน้อย 8 cores (แนะนำ 16 cores หรือมากกว่า)
- 1.2) RAM: อย่างน้อย 32 GB (แนะนำ 64 GB หรือมากกว่า)
- 1.3) Storage: SSD อย่างน้อย 500 GB สำหรับ OS และ Applications
- 1.4) Storage: HDD/SSD เพิ่มเติมสำหรับ VM Storage
- 1.5) Network: การเชื่อมต่อเครือข่ายความเร็วสูงและเสถียร

2) การติดตั้ง Proxmox VE:

- 2.1) ดาวน์โหลด Proxmox VE ISO จาก <https://www.proxmox.com/>
- 2.2) ติดตั้งบน Server Hardware หรือ Virtual Machine
- 2.3) ตั้งค่า Network และ Storage Pool
- 2.4) สร้าง VM Templates สำหรับการใช้งาน

3) การติดตั้งซอฟต์แวร์พื้นฐาน:

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Install Docker Compose
sudo apt install docker-compose-plugin

# Install Bun
curl -fsSL https://bun.sh/install | bash
```

ภาพที่ ค-1 การติดตั้งซอฟต์แวร์พื้นฐานสำหรับ Production

ค.2 การตั้งค่า Environment Variables สำหรับ Production

การตั้งค่า Environment Variables ที่เหมาะสมสำหรับการใช้งานจริง

ไฟล์ .env สำหรับ Production

```
# Database Configuration
DB_HOST=localhost
DB_PORT=5432
DB_USER=cloudplatform
DB_PASSWORD={strong_password_here}
DB_NAME=cloudplatform_prod

# RabbitMQ Configuration
RABBITMQ_HOST=localhost
RABBITMQ_PORT=5672
RABBITMQ_USER=cloudplatform
RABBITMQ_PASSWORD={strong_password_here}

# Proxmox Configuration
PROXMOX_HOST={proxmox_server_ip}
PROXMOX_USERNAME={api_username}
PROXMOX_PASSWORD={api_password}
PROXMOX_NODE={proxmox_node_name}

# Application Ports
MIDORI_PORT=3000
MOMOI_PORT=3001

# Security
JWT_SECRET={very_strong_jwt_secret}
ENCRYPTION_KEY={32_char_encryption_key}

# Monitoring
JAEGER_ENDPOINT=http://localhost:14268/api/traces
PROMETHEUS_PORT=9090
LOKI_ENDPOINT=http://localhost:3100
```

ภาพที่ ค-2 ตัวอย่างการตั้งค่า Environment Variables สำหรับ Production

ค.3 การ Deploy ด้วย Docker Compose

การใช้ Docker Compose สำหรับการ Deploy ระบบแบบ Production

การสร้าง Production Docker Compose

- 1) สร้างไฟล์ `docker-compose.prod.yml`
- 2) รันคำสั่งสำหรับ Deploy

```

version: '3.8'

services:
  midori:
    build:
      context: .
      dockerfile: docker/midori.dockerfile
    ports:
      - "${MIDORI_PORT:-3000}:3000"
    environment:
      - NODE_ENV=production
      - MOMOI_API_URL=http://momoi:3001
    depends_on:
      - momoi
    restart: unless-stopped

  momoi:
    build:
      context: .
      dockerfile: docker/momoi.dockerfile
    ports:
      - "${MOMOI_PORT:-3001}:3001"
    environment:
      - NODE_ENV=production
    depends_on:
      - db
      - queue
    restart: unless-stopped

```

ภาพที่ ค-3 ตัวอย่าง Docker Compose สำหรับ Production

```

# Build    Start Services
docker-compose -f docker-compose.prod.yml up -d --build

#          Services
docker-compose -f docker-compose.prod.yml ps

#  Logs
docker-compose -f docker-compose.prod.yml logs -f

```

ภาพที่ ค-4 คำสั่งสำหรับ Deploy ด้วย Docker Compose

ค.4 การตั้งค่า Reverse Proxy และ SSL

การตั้งค่า Nginx เป็น Reverse Proxy และการติดตั้ง SSL Certificate

การติดตั้งและตั้งค่า Nginx

1) ติดตั้ง Nginx

```
sudo apt install nginx
sudo systemctl enable nginx
sudo systemctl start nginx
```

ภาพที่ ค-5 การติดตั้ง Nginx

2) สร้างไฟล์ Configuration สำหรับ Site

```
server {
    listen 80;
    server_name your-domain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    location /api {
        proxy_pass http://localhost:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

ภาพที่ ค-6 การตั้งค่า Nginx Reverse Proxy

3) ติดตั้ง SSL Certificate ด้วย Let's Encrypt

```
# Certbot
sudo apt install certbot python3-certbot-nginx

# SSL Certificate
sudo certbot --nginx -d your-domain.com

# Auto-renewal
sudo crontab -e
# : 0 12 * * * /usr/bin/certbot renew --quiet
```

ภาพที่ ค-7 การติดตั้ง SSL Certificate

ค.5 การตั้งค่า Database สำหรับ Production

การเตรียม PostgreSQL Database สำหรับการใช้งานจริง

การตั้งค่า PostgreSQL

1) การตั้งค่า Database User และ Permissions

```
# PostgreSQL
sudo -u postgres psql

# Database User
CREATE DATABASE cloudplatform_prod;
CREATE USER cloudplatform WITH ENCRYPTED PASSWORD
'strong_password';
GRANT ALL PRIVILEGES ON DATABASE cloudplatform_prod TO
cloudplatform;

# PostgreSQL
q
```

ภาพที่ ค-8 การตั้งค่า PostgreSQL Database

2) การรัน Database Migration

3) การตั้งค่า Database Backup

ค.6 การตรวจสอบและบำรุงรักษาระบบ

การตรวจสอบและบำรุงรักษาระบบเพื่อให้ทำงานได้อย่างเสถียร

การตรวจสอบสถานะระบบ

```
# Momoi directory
cd apps/momoi

# Prisma Migration
bunx prisma migrate deploy

# Generate Prisma Client
bunx prisma generate
```

ภาพที่ ค-9 การรัน Database Migration

```
#!/bin/bash
# backup-db.sh

DB_NAME="cloudplatform_prod"
DB_USER="cloudplatform"
BACKUP_DIR="/backups/postgresql"
DATE=$(date +%Y%m%d_%H%M%S)

# directory
mkdir -p $BACKUP_DIR

# Database
pg_dump -h localhost -U $DB_USER -d $DB_NAME >
$BACKUP_DIR/backup_$DATE.sql

# backup 7
find $BACKUP_DIR -name "backup_*.sql" -mtime +7 -delete
```

ภาพที่ ค-10 สคริปต์สำหรับ Database Backup

1) สร้างสคริปต์ตรวจสอบสถานะ Services

```
#!/bin/bash
# health-check.sh

echo "=== System Health Check ==="

# Docker Services
echo "Docker Services Status:"
docker-compose -f docker-compose.prod.yml ps

# Nginx
echo "Nginx Status:"
systemctl status nginx --no-pager

# Disk Space
echo "Disk Usage:"
df -h

# Memory
echo "Memory Usage:"
free -h

# Database
echo "Database Connection:"
pg_isready -h localhost -p 5432
```

ภาพที่ ค-11 สคริปต์ตรวจสอบสถานะระบบ

2) ตั้งค่า Cron Job สำหรับการตรวจสอบอัตโนมัติ

การอัปเดตระบบ

1) การอัปเดตแอปพลิเคชัน

ค.7 การแก้ไขปัญหาใน Production

วิธีการแก้ไขปัญหาที่อาจเกิดขึ้นในการใช้งานจริง

ปัญหาที่พบบ่อยและวิธีแก้ไข

1) Service ไม่ตอบสนอง

1.1) ตรวจสอบ Logs: `docker-compose logs [service_name]`

```
#    Crontab
crontab -e

#           5
*/5 * * * * /path/to/health-check.sh >> /var/log/health-check.log

#    Database      2:00 .
0 2 * * * /path/to/backup-db.sh

#           Docker images
0 3 * * 0 docker system prune -f
```

ภาพที่ ค-12 การตั้งค่า Cron Jobs

```
#!/bin/bash
#    update-app.sh

echo "Starting application update..."

# Pull latest code
git pull origin main

# Install dependencies
bun install

# Build    restart services
docker-compose -f docker-compose.prod.yml down
docker-compose -f docker-compose.prod.yml up -d --build

#           update
sleep 30
docker-compose -f docker-compose.prod.yml ps

echo "Update completed!"
```

ภาพที่ ค-13 สคริปต์สำหรับอัปเดตแอปพลิเคชัน

- 1.2) รีสตาร์ท Service: `docker-compose restart [service_name]`
- 1.3) ตรวจสอบ Resource Usage: `docker stats`
- 2) **Database Connection Error**
 - 2.1) ตรวจสอบ PostgreSQL Service: `systemctl status postgresql`
 - 2.2) ตรวจสอบ Connection Pool: ดู Logs ของ Momoi Service
 - 2.3) รีสตาร์ท Database: `sudo systemctl restart postgresql`
- 3) **Out of Disk Space**
 - 3.1) ทำความสะอาด Docker: `docker system prune -a`
 - 3.2) ลบ Log files เก่า: `find /var/log -name "*.log" -mtime +30 -delete`
 - 3.3) ตรวจสอบ VM Storage ใน Proxmox
- 4) **SSL Certificate หมดอายุ**
 - 4.1) ต่ออายุ Certificate: `sudo certbot renew`
 - 4.2) รีโหลด Nginx: `sudo nginx -s reload`
 - 4.3) ตรวจสอบวันหมดอายุ: `sudo certbot certificates`